

Project 1

ECE 232E Summer 2026 - Project 1: Random Graphs and Random Walks

Posted: Friday, June 26, 2026

Due: Sunday, July 12, 2026 at 11:59 PM PST

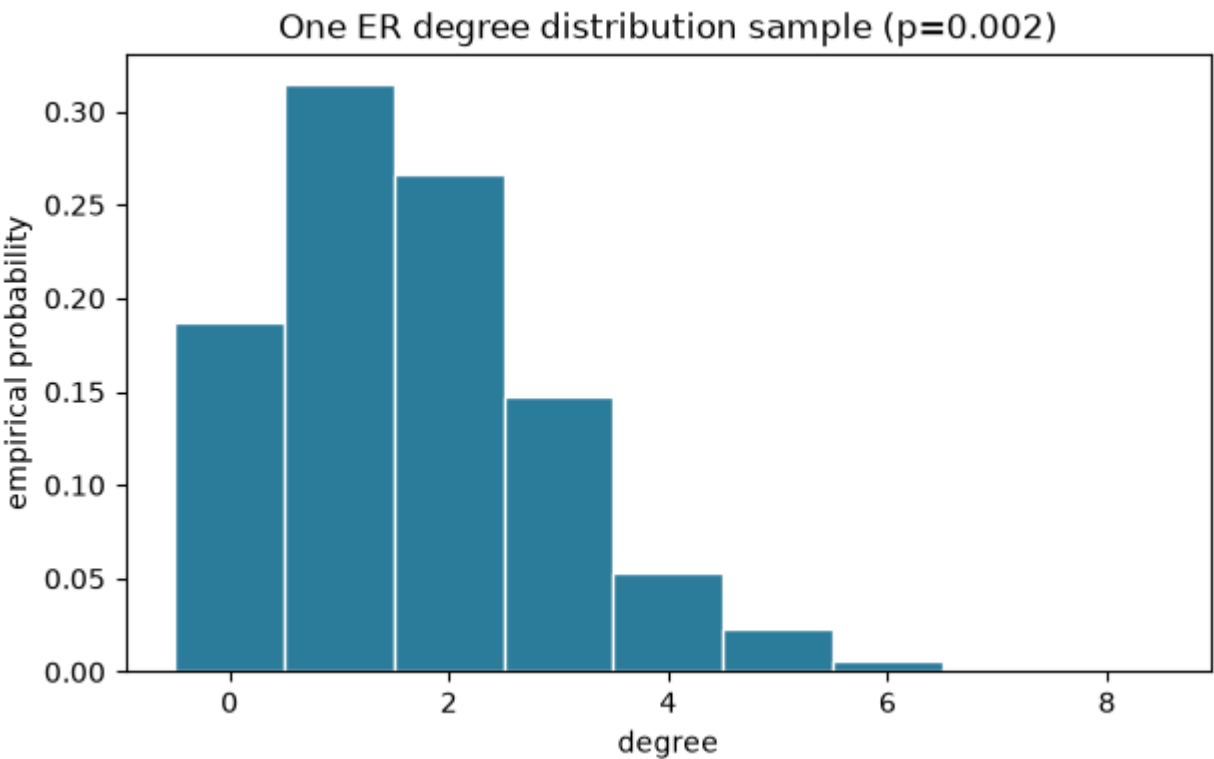
By Enoch Yeoh 906630606 / Nicolas Jeong Lee 605309755 / James Chen 005399315

1. Generating Random Networks

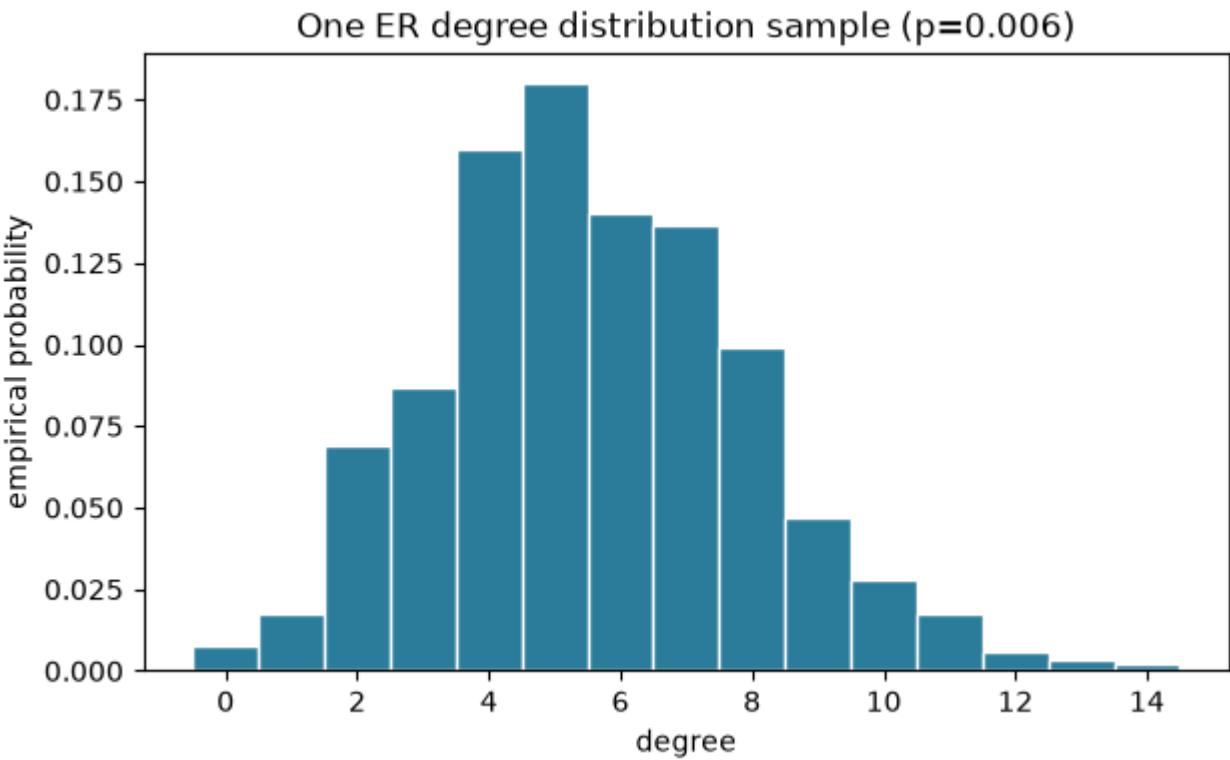
1. Create random networks using Erdos-Renyi (ER) model

(a)

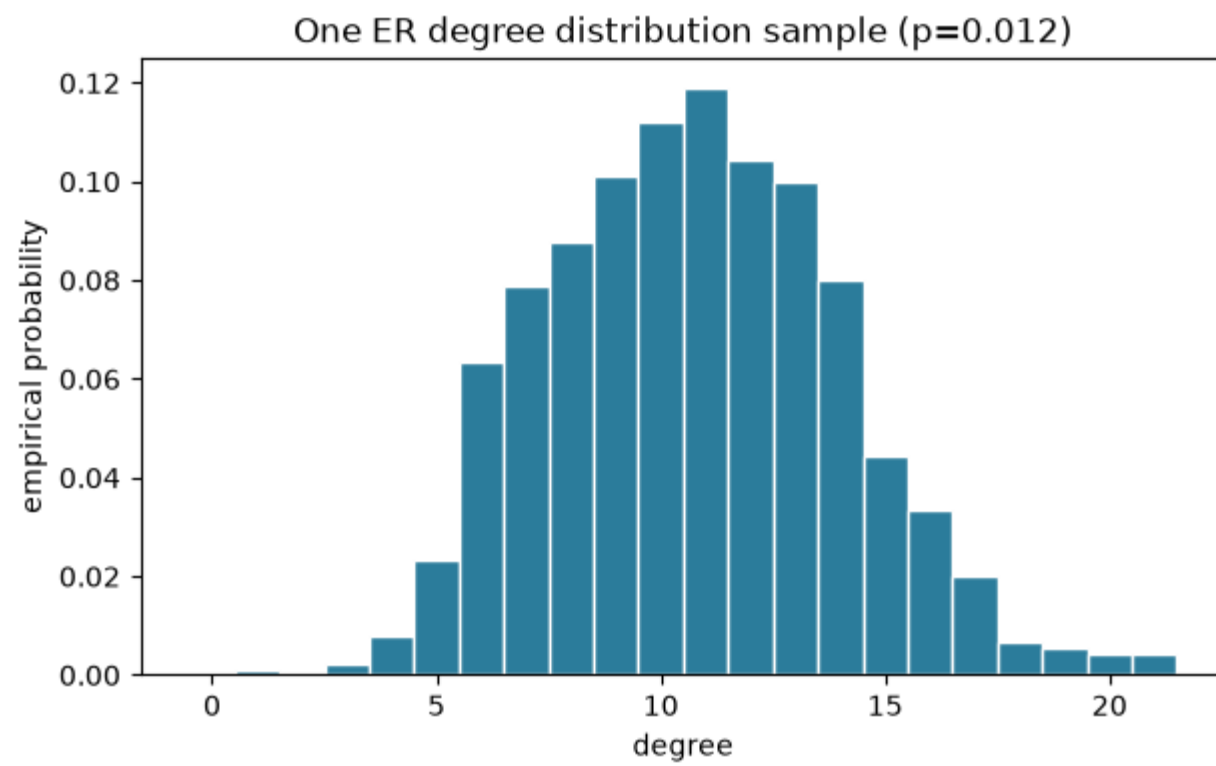
```
In [24]: from project1_code import question_1a
question_1a()
```



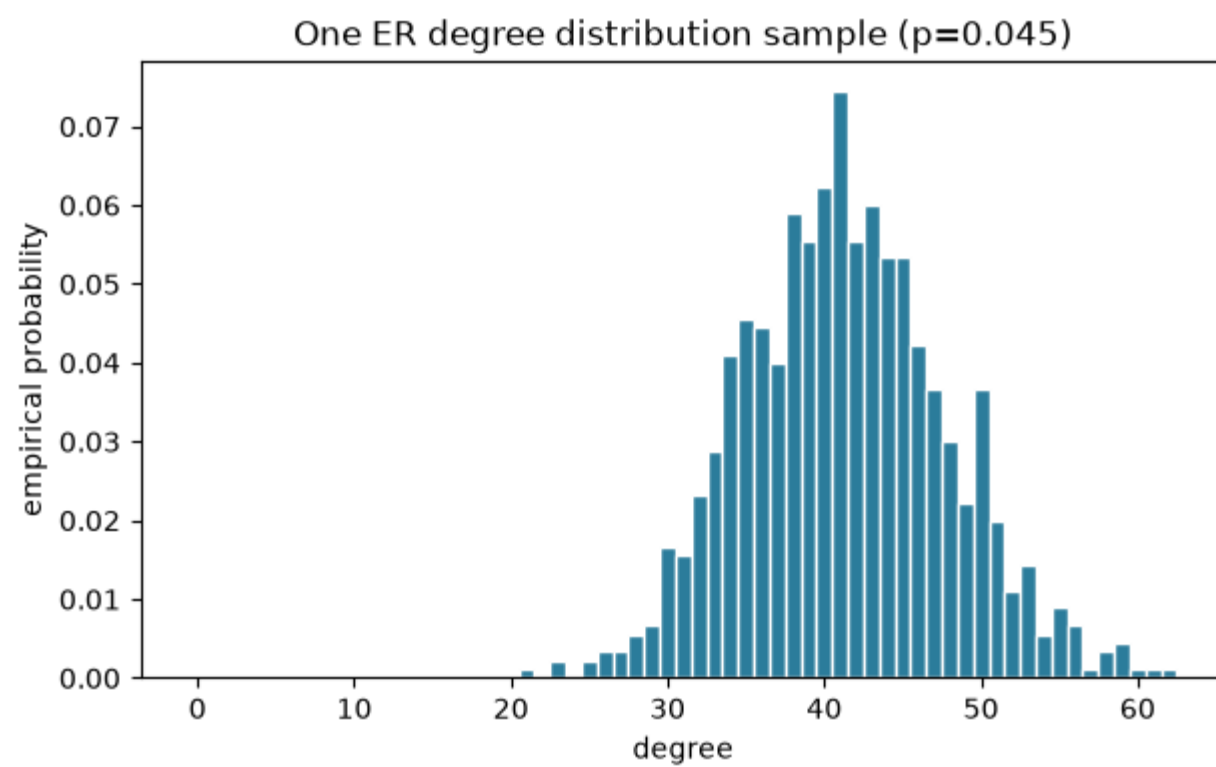
empirical mean: 1.671111111111112
theory mean: 1.798
empirical variance: 1.680720987654321
theory variance: 1.794404



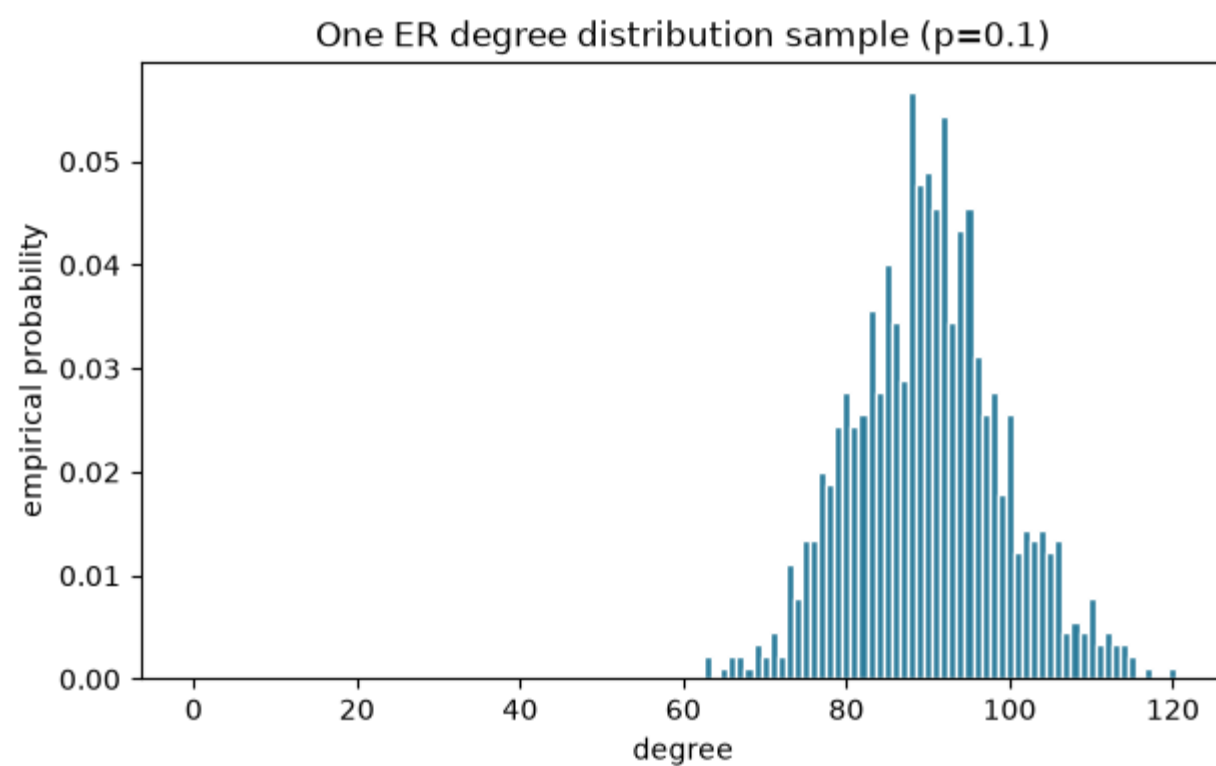
empirical mean: 5.577777777777775
theory mean: 5.394
empirical variance: 5.595061728395062
theory variance: 5.361636



empirical mean: 10.751111111111111
 theory mean: 10.788
 empirical variance: 10.540276543209876
 theory variance: 10.658544000000001



empirical mean: 41.291111111111114
 theory mean: 40.455
 empirical variance: 41.977476543209875
 theory variance: 38.634525



empirical mean: 89.98666666666666
 theory mean: 89.9
 empirical variance: 84.74648888888889
 theory variance: 80.910000000000001

We observe a Gaussian distribution that starts centered at low degree values near the origin (when the probability p is low) and gradually shifts to the right (when p grows).

Since the degree distribution follows a binomial distribution, the expected value (peak) has the relationship $n * p$, where n is the number of

nodes. Since n is fixed, an increased p value would shift the center of the distribution progressively to the right, which is observed in our plots.

A binomial distribution increasingly resembles a Gaussian distribution if $n * p$ is sufficiently large. The rule of thumb is often $n * p \geq 5$ and $n * (1-p) \geq 5$. We observe our distributions resemble a Gaussian one as p increases.

(b)

```
In [25]: from project1_code import question_1b
question_1b()
```

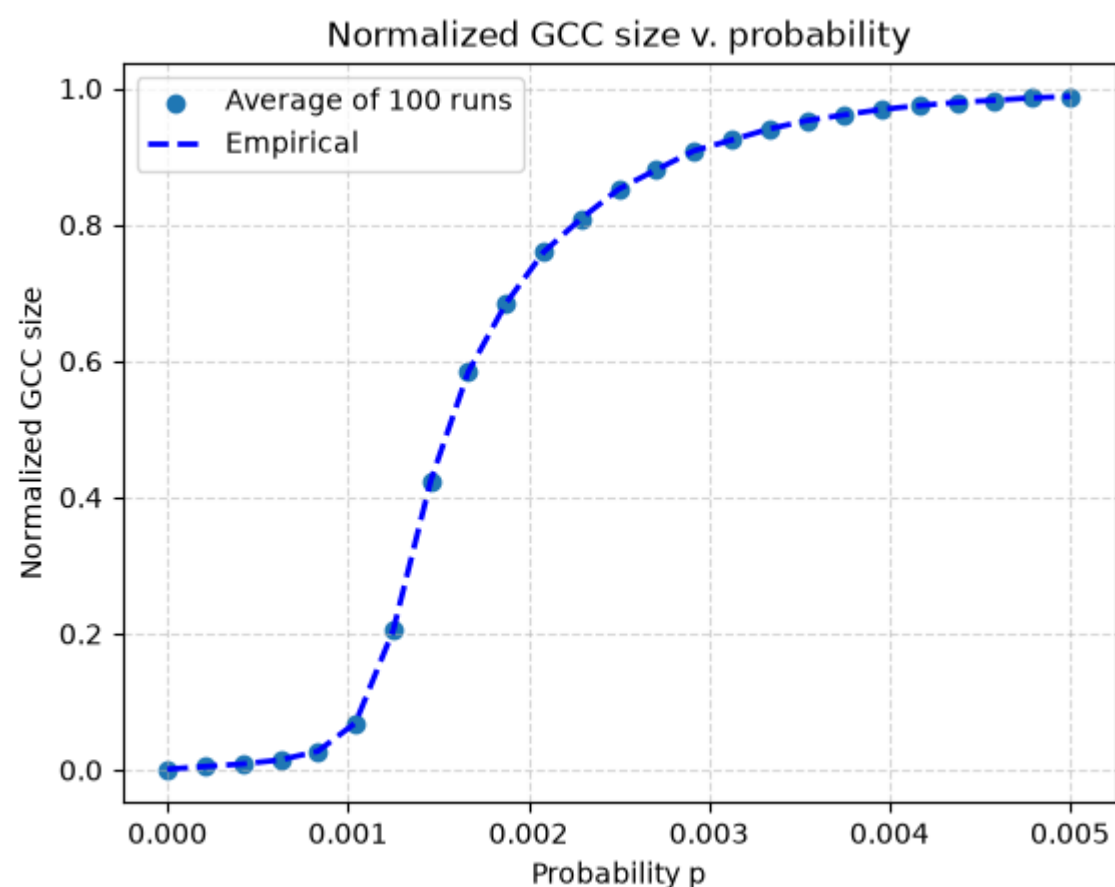
```
p=0.002 P(conn)~0.00, avg GCC frac~0.73, avg GCC diam~25.30
p=0.006 P(conn)~0.00, avg GCC frac~1.00, avg GCC diam~8.20
p=0.012 P(conn)~1.00, avg GCC frac~1.00, avg GCC diam~5.00
p=0.045 P(conn)~1.00, avg GCC frac~1.00, avg GCC diam~3.00
p=0.100 P(conn)~1.00, avg GCC frac~1.00, avg GCC diam~3.00
GCC diameter=24 for a disconnected network (p=0.002) instance
```

All random realizations of the ER network are not connected. As the probability p increases, however, the probability of connectivity increases.

For one instance of a disconnected, randomly generated network with $p = 0.002$, the GCC diameter = 22. By definition, a Giant Connected Network (GCC) is a subset, or a subgraph, of the graph that we randomly generate and is always connected and hence, a non-infinite and numerical diameter answer.

(c)

```
In [1]: from project1_code import question_1c
question_1c()
```



```
First X value above Y > 0.005: 0.00021
Actual Y value at that point: 0.00520
First X value above Y > 0.98: 0.00458
Actual Y value at that point: 0.98256
```

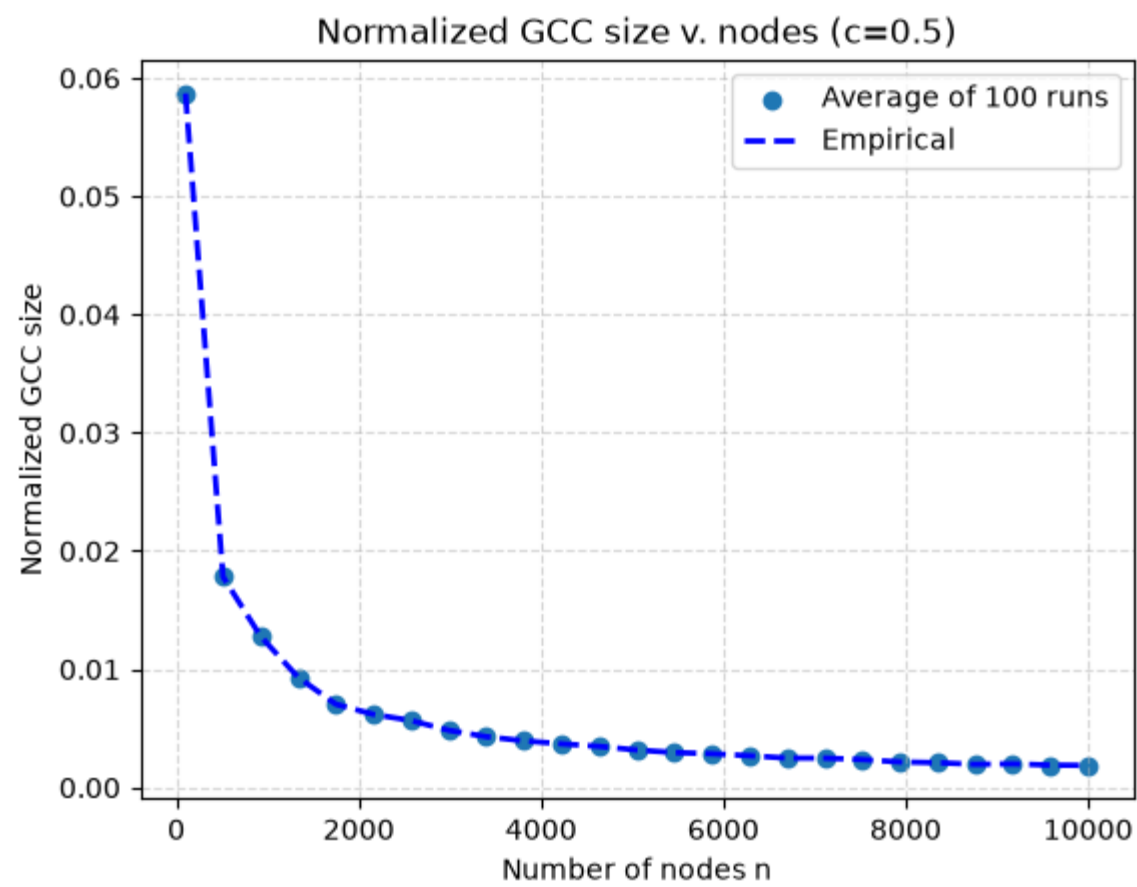
```
Part (i) p = 0.00021, gcc_size = 0.00520
Part (ii) p = 0.00458, gcc_size = 0.98256
```

i. A GCC starts to emerge at $p \sim 0.001$. Yes, this matches the theoretical value mentioned in lecture, whereby a GCC emerges when the $np=1$ equation is satisfied. That is $p = 1/n = 1/900 = 0.0011$

ii. For $p > 0.005$, the GCC size takes up over 99% of the nodes in almost every experiment.

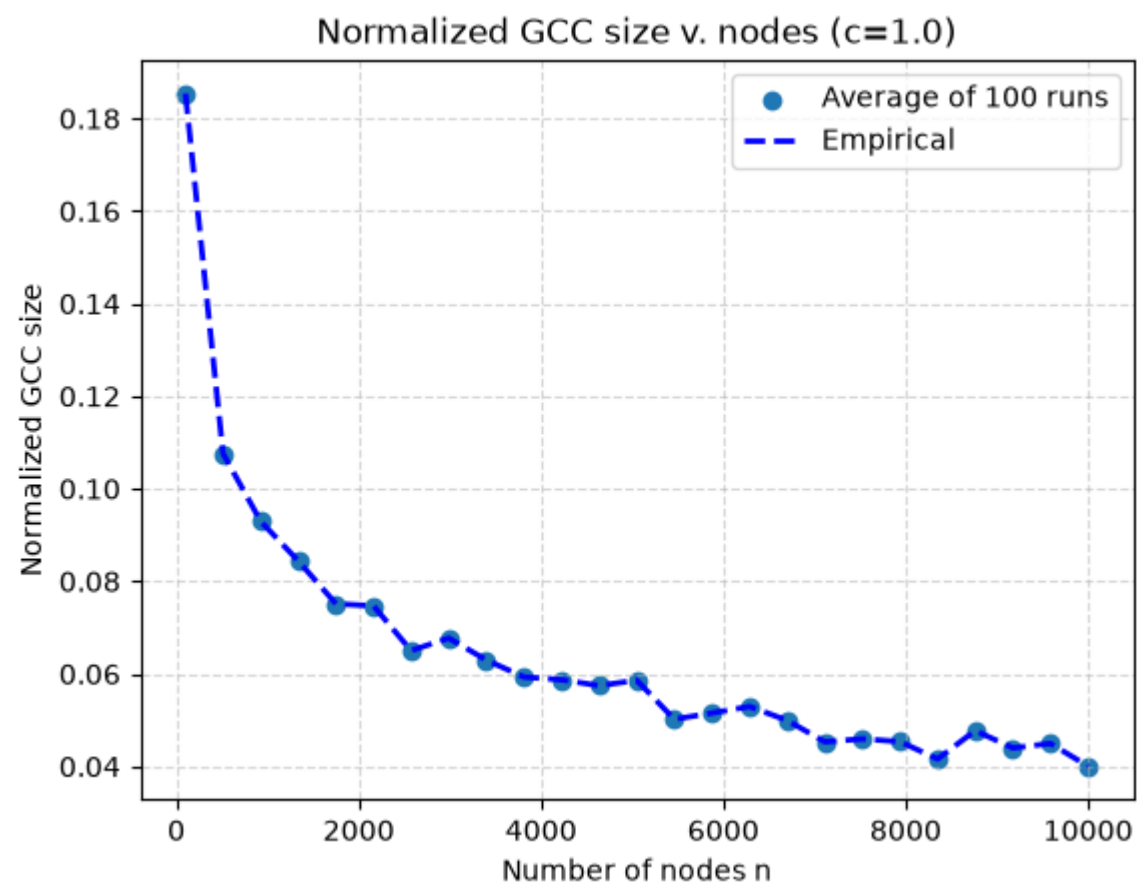
(d)

```
In [2]: from project1_code import question_1d
question_1d(c=0.5)
```



i. We observe a decreasing exponential trend as the number of nodes increases. As n increases, the edge-formation probability decreases due to the $p=c/n$ relationship, which results in decreased GCC size (indicating an increasingly fractured network) that approaches zero.

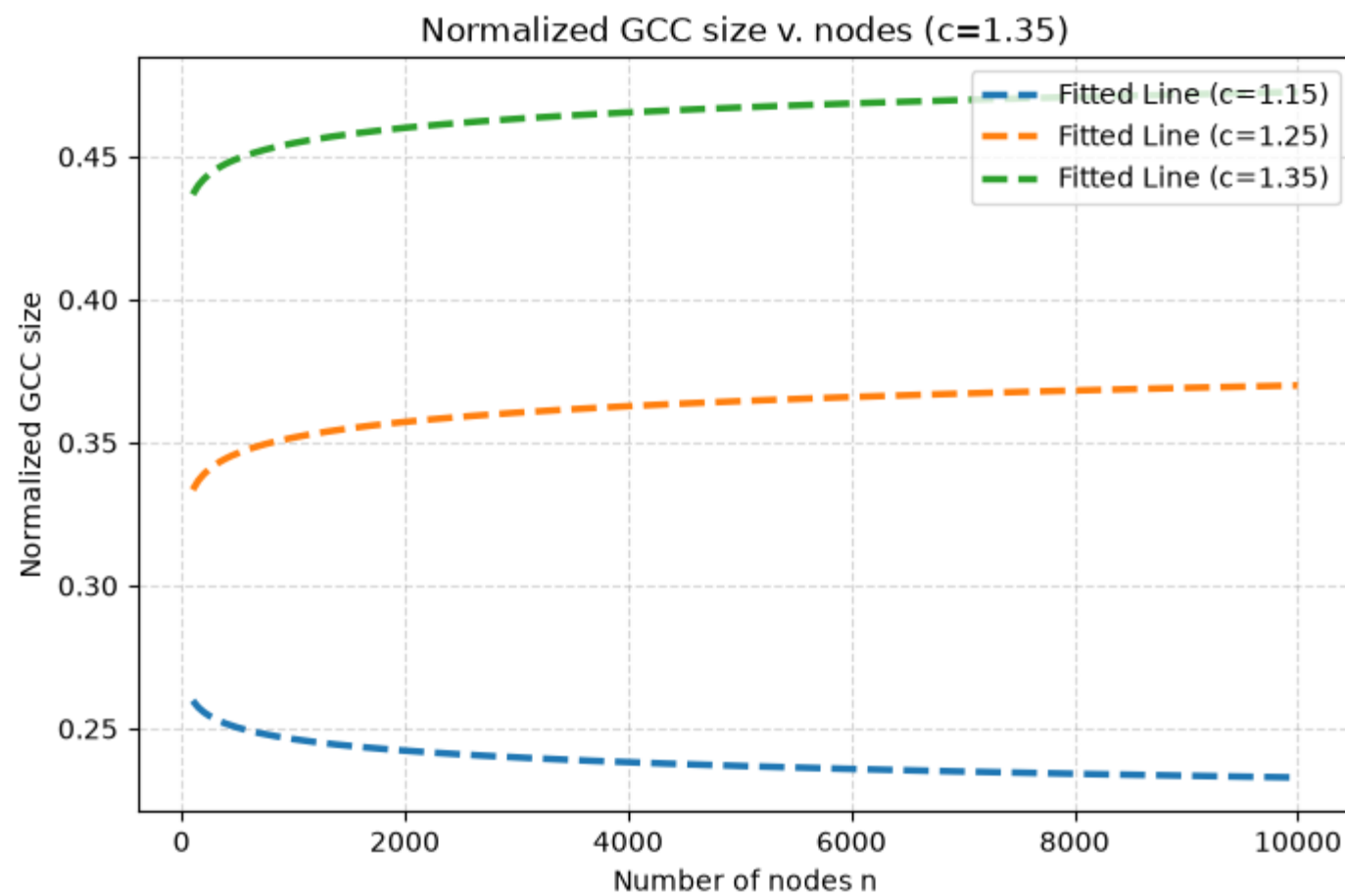
In [3]: `question_1d(c=1.0)`



ii. The same decreasing exponential trend is observed for $c=1$, except the GCC size does not approach zero but remains on a higher floor centering 0.04

In []: `from project1_code import question_1d_part_iv`
`question_1d_part_iv()`

Generating random networks for $c=1.15$
 Generating random networks for $c=1.25$
 Generating random networks for $c=1.35$



iii. See above for $c=1.15$, 1.25 , 1.35 plots.

iv. As c increases, we observe a logarithmic trend. As the number of nodes increase, the GCC size increases and saturates to a steady state value. The steady-state value becomes higher as c , the average degree of nodes, increases. For $c=1.15$ in particular, the trend line starts to, but not quite yet, resemble a logarithmic pattern, having passed the critical $c=1$ threshold when many isolated components of the network start to connect. The logarithmic trend is much more evident with $c=1.25$ and $c=1.35$.

2. Create networks using preferential attachment model

(a)

```
In [ ]: from project1_code import question_2a
question_2a()
```

```
[True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True]
Yes, this network is always connected
```

(b)

```
In [ ]: from project1_code import question_2b
question_2b()
```

```
IGRAPH U--- 1050 1049 --
connected: True
max degree: 24
mean degree: 1.9980952380952381
degree variance: 4.17332970521542
edges: 1049 , assortativity: -0.09135136092382576 , modularity: 0.935
Assortativity is defined as a measure of the tendency for nodes to connect to other nodes with similar properties
Assortativity is positive if similar nodes tend to connect to each other, and negative otherwise
```

(c)

```
In [ ]: from project1_code import question_2c
question_2c()
```

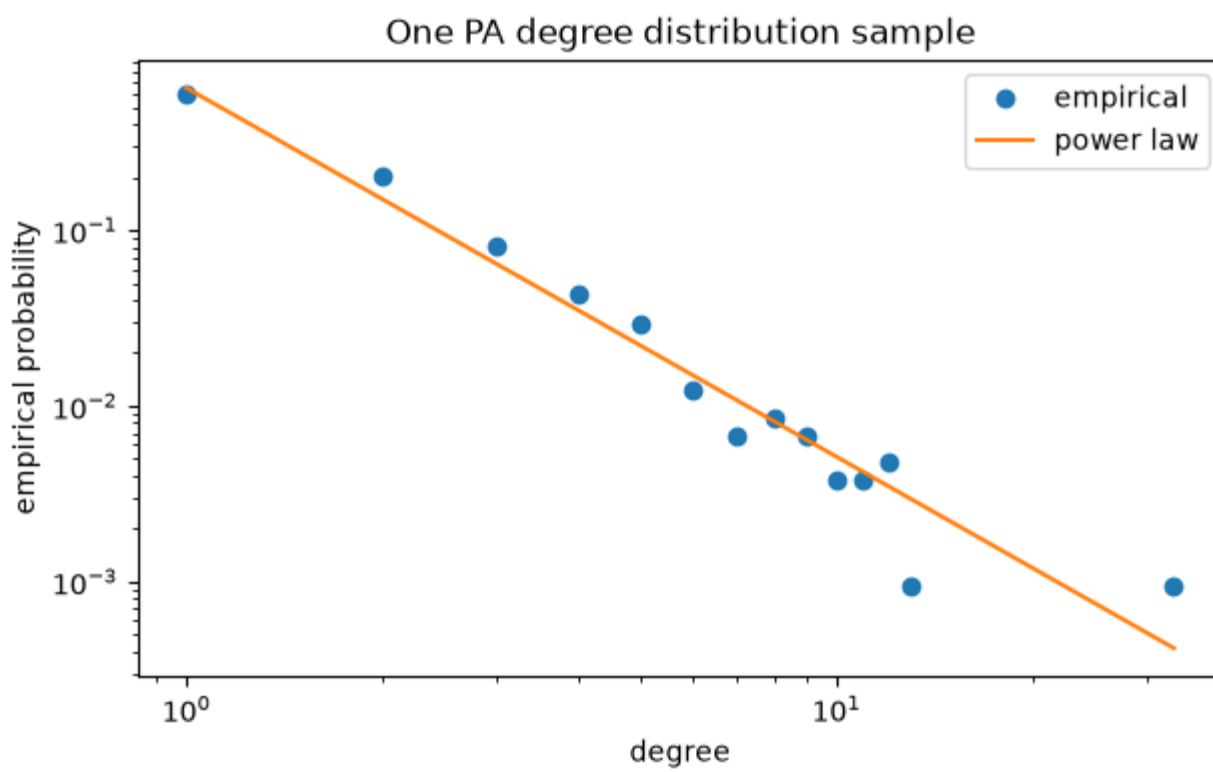
```
IGRAPH U--- 10500 10499 --
connected: True
max degree: 95
mean degree: 1.9998095238095237
degree variance: 5.846476154195011
edges: 10499 , assortativity: -0.03100035425224403 , modularity: 0.979
```

The value of assortativity for a 10500 nodes is slightly less negative and the value of modularity is slightly higher than the one with 1050 nodes. However, the change in value isn't that big to claim that there is a significant difference.

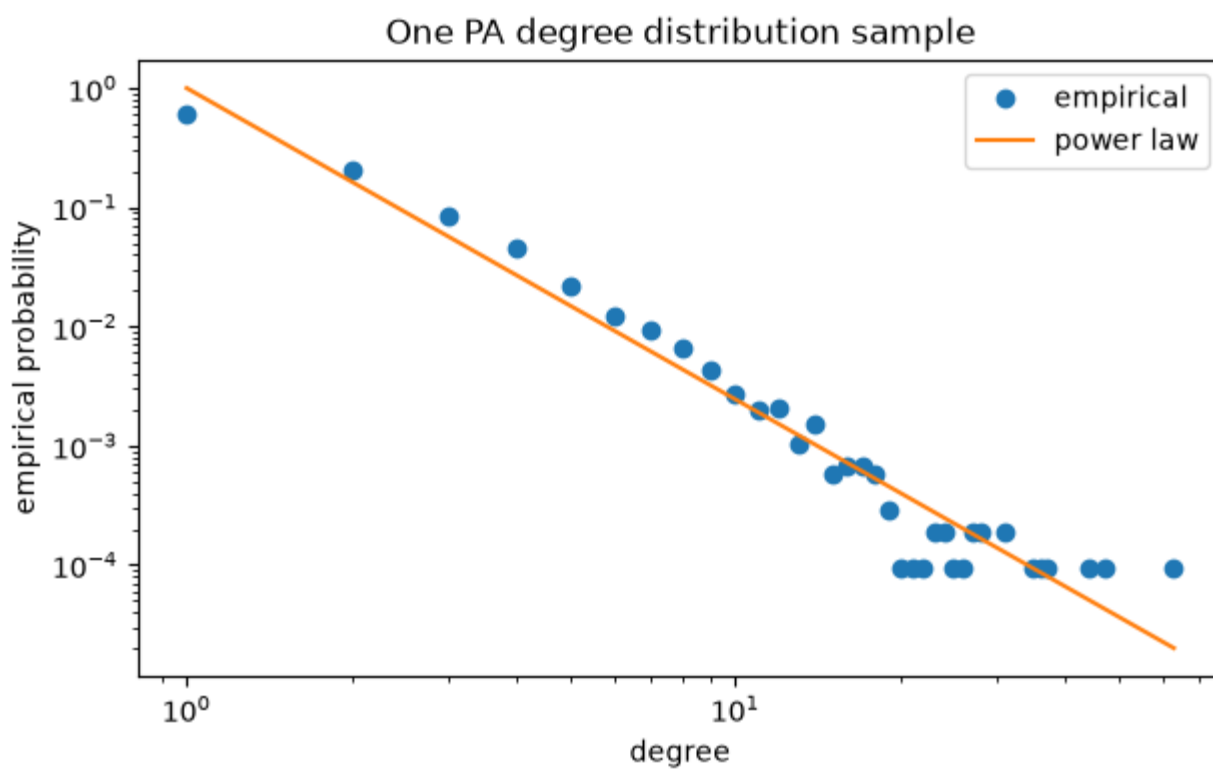
(d)

```
In [ ]: from project1_code import question_2d
question_2d()
```

```
Nodes: 1050 , m = 1
rough log-log slope: -2.0998976217006526
```



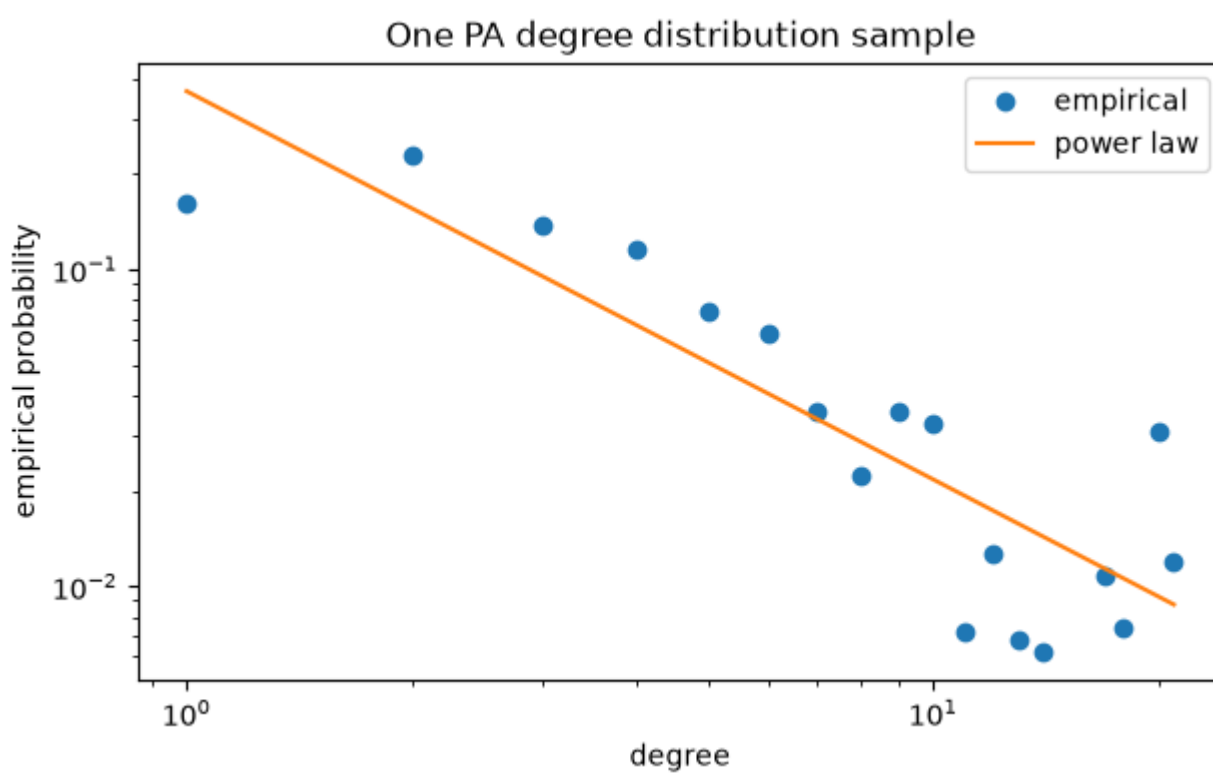
Nodes: 10500 , m = 1
rough log-log slope: -2.6111392722663567



(e)

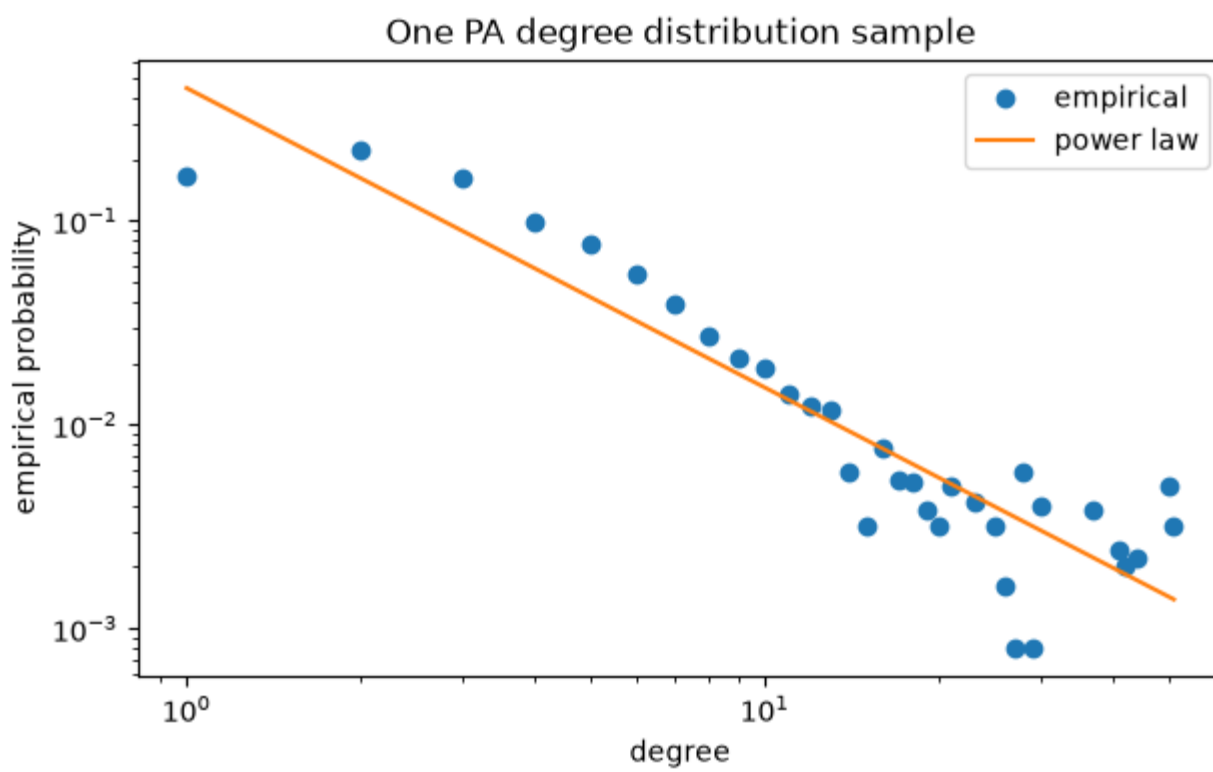
```
In [ ]: from project1_code import question_2e
question_2e(1050)
```

rough log-log slope: -1.2256956531976575



```
In [ ]: question_2e(10500)
```

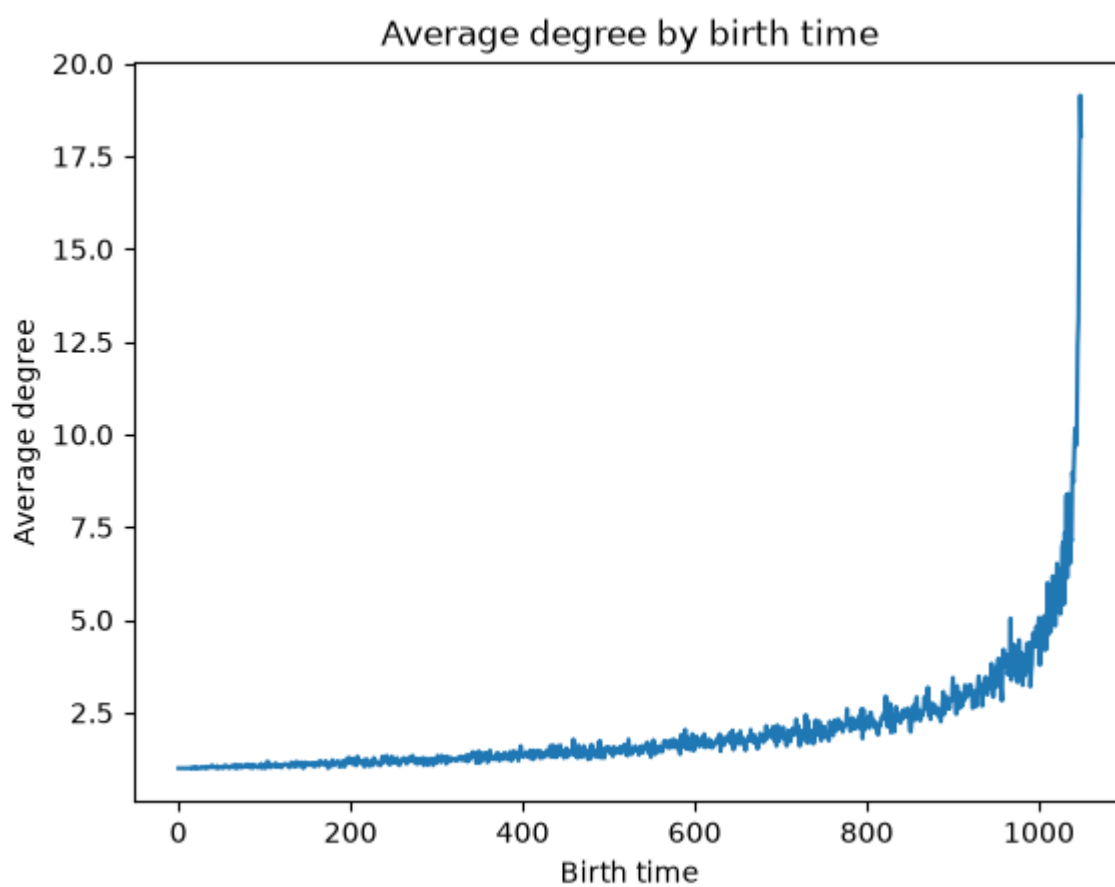
rough log-log slope: -1.4686951209862034



The distribution should still be linear since sampling a neighbor of a random node is still fundamentally drawing from a degree-biased version of the same power-law population. That being said, the slope becomes less negative(or less steep) compared to node degree distribution.

(f)

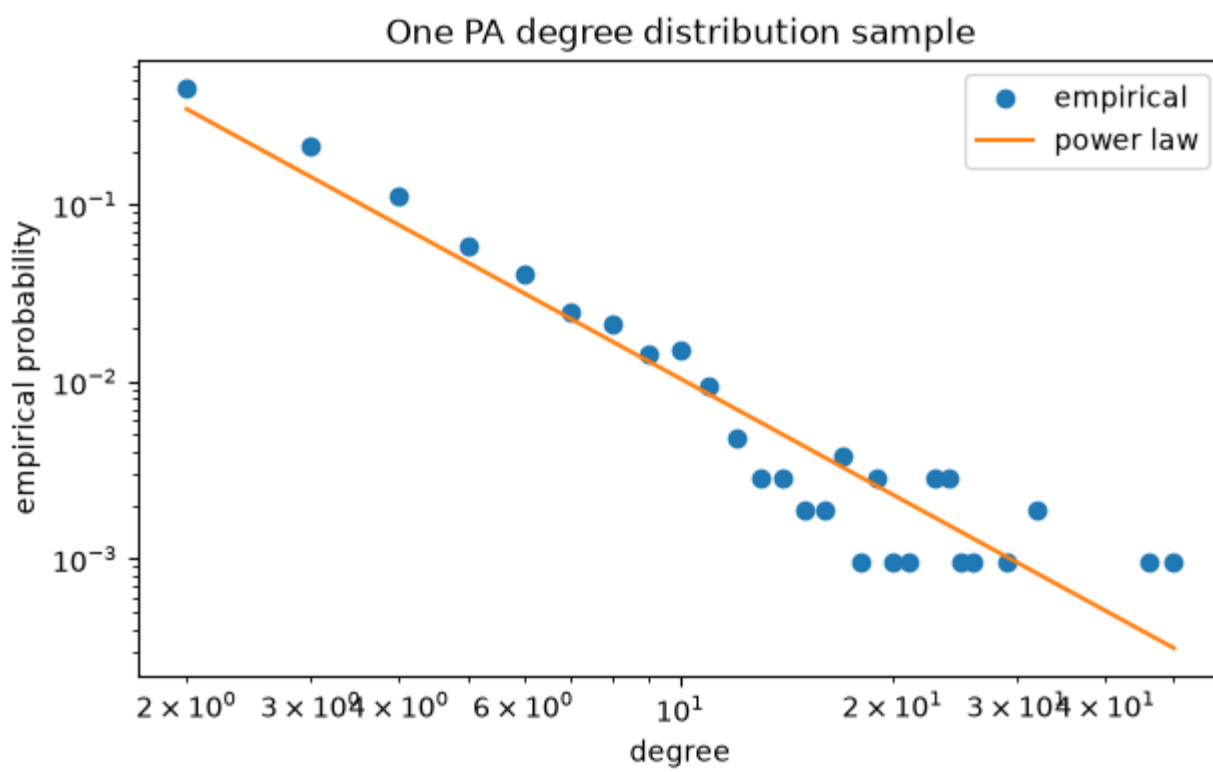
```
In [ ]: from project1_code import question_2f
question_2f()
```



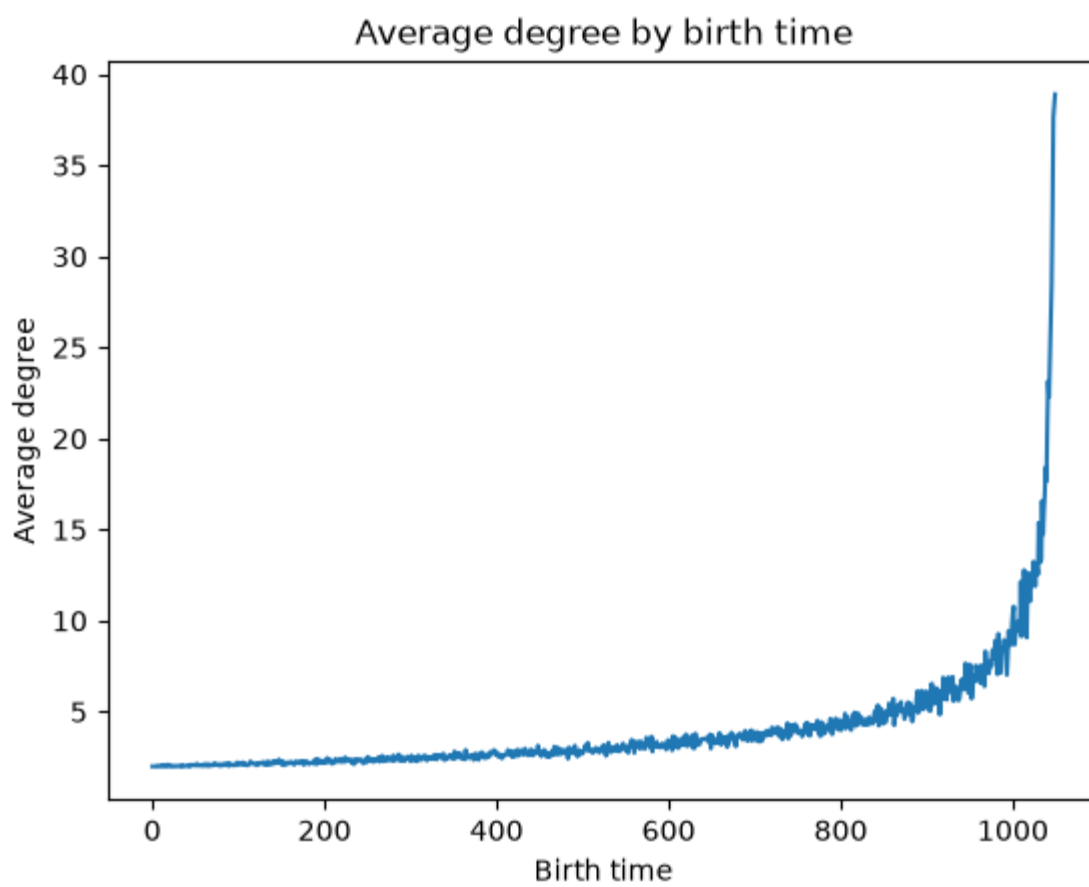
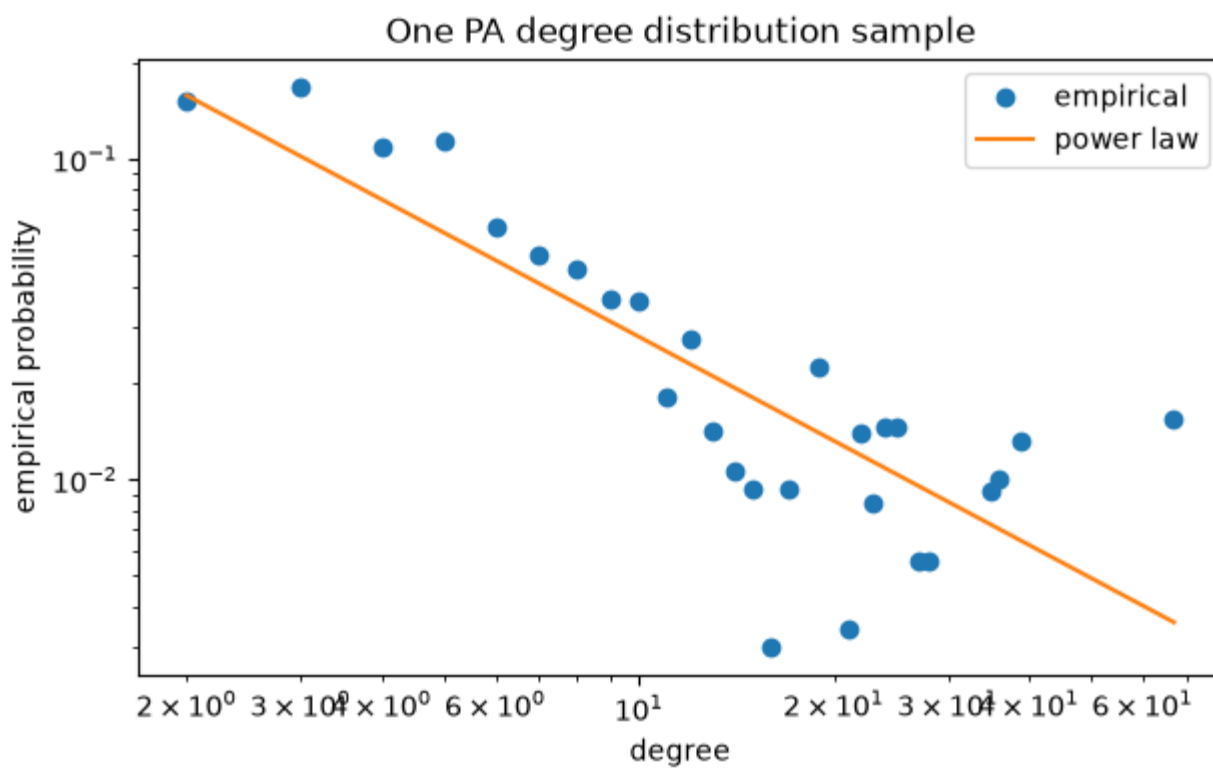
(g)

```
In [ ]: from project1_code import question_2g
question_2g(2)
```

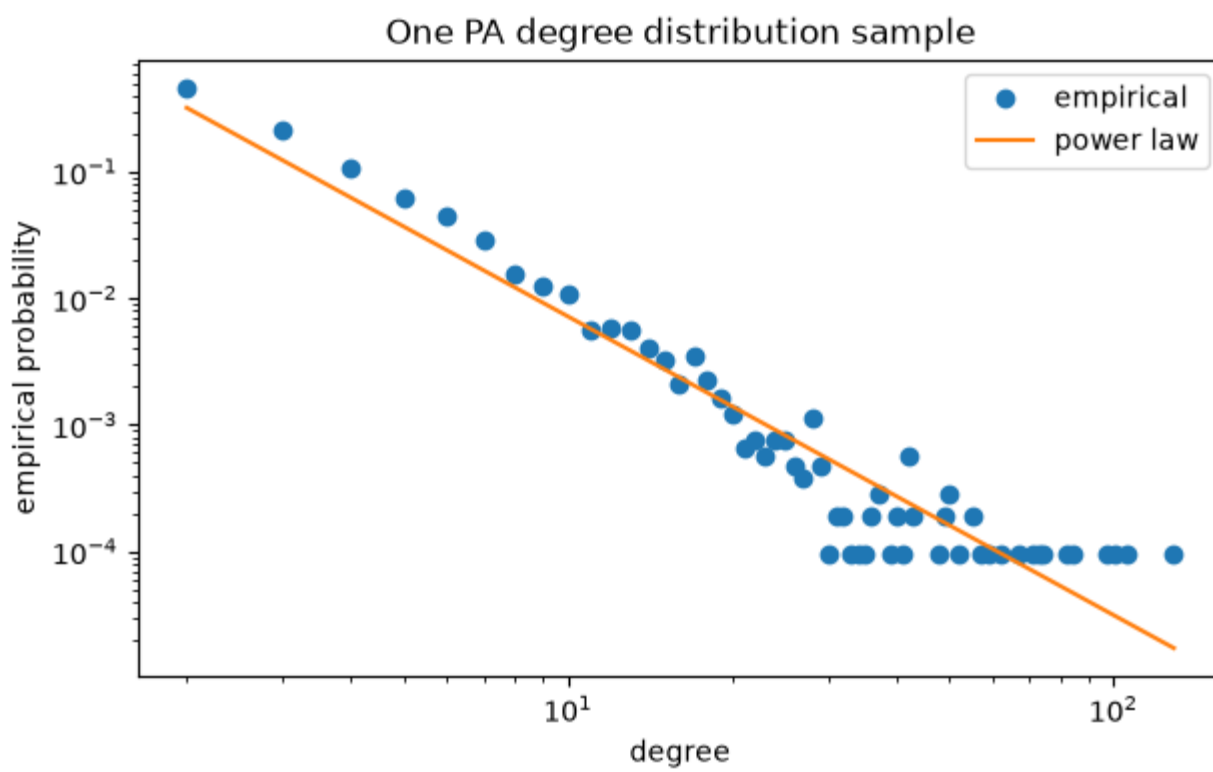
```
[True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True]
Yes, this network is always connected
IGRAPH U--- 1050 2097 --
connected: True
max degree: 60
mean degree: 3.994285714285714
degree variance: 17.937110204081634
edges: 2097 , assortativity: -0.04381372807349328 , modularity: 0.522
Nodes: 1050 , m = 2
rough log-log slope: -2.177362601647715
```



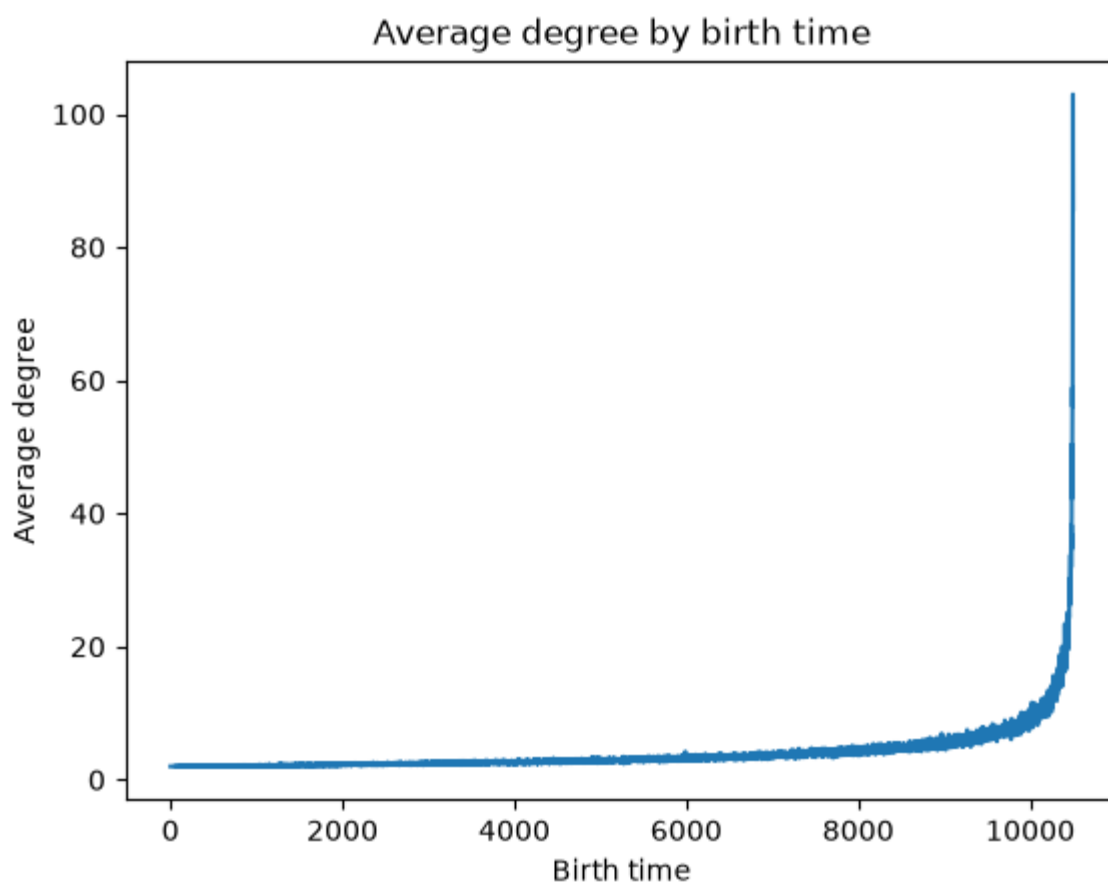
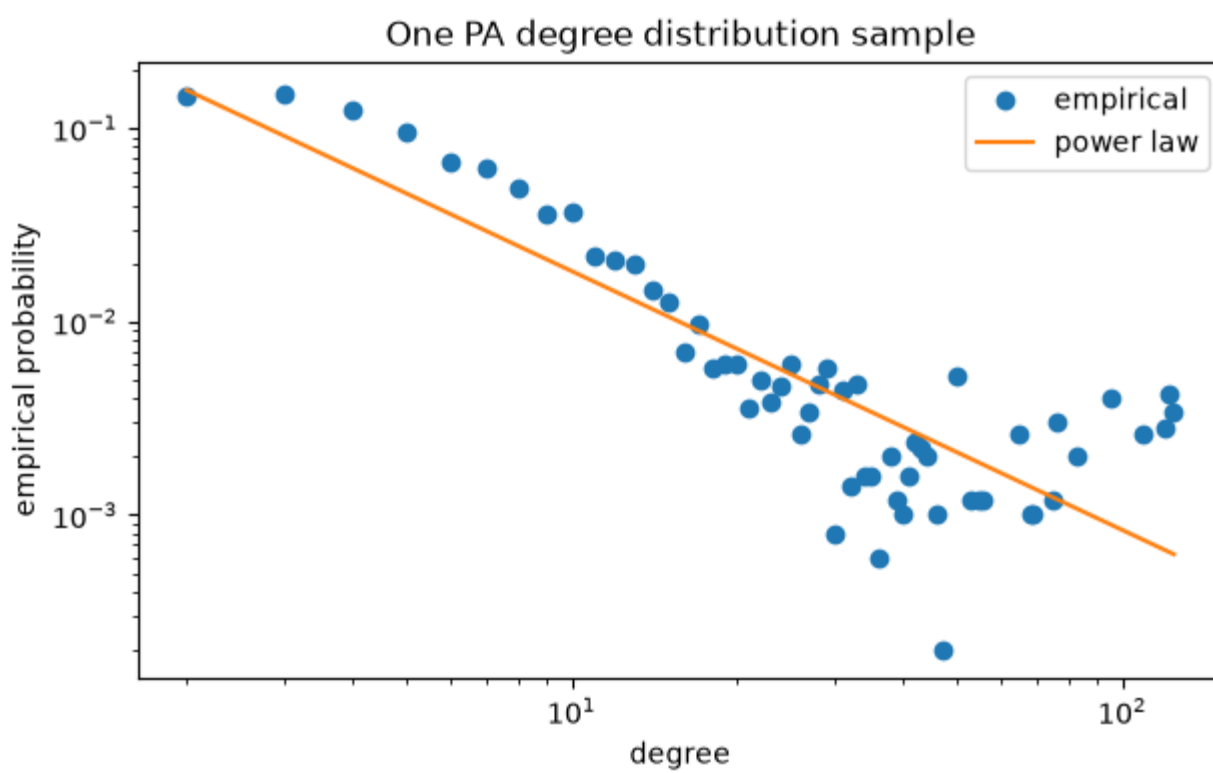
rough log-log slope: -1.077340553118729



IGRAPH U--- 10500 20997 --
 connected: True
 max degree: 139
 mean degree: 3.9994285714285716
 degree variance: 23.31619014965986
 edges: 20997 , assortativity: -0.010584339608166836 , modularity: 0.532
 Nodes: 10500 , m = 2
 rough log-log slope: -2.3593799931878996

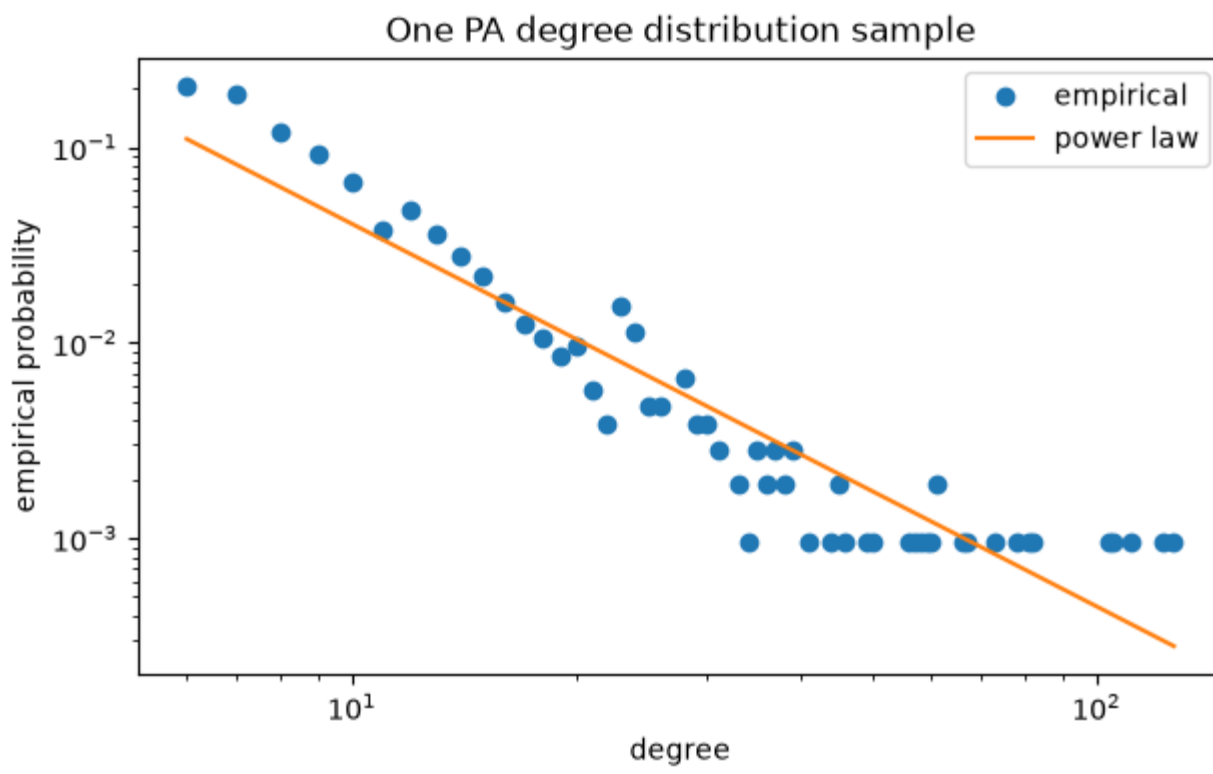


rough log-log slope: -1.3391496625798234

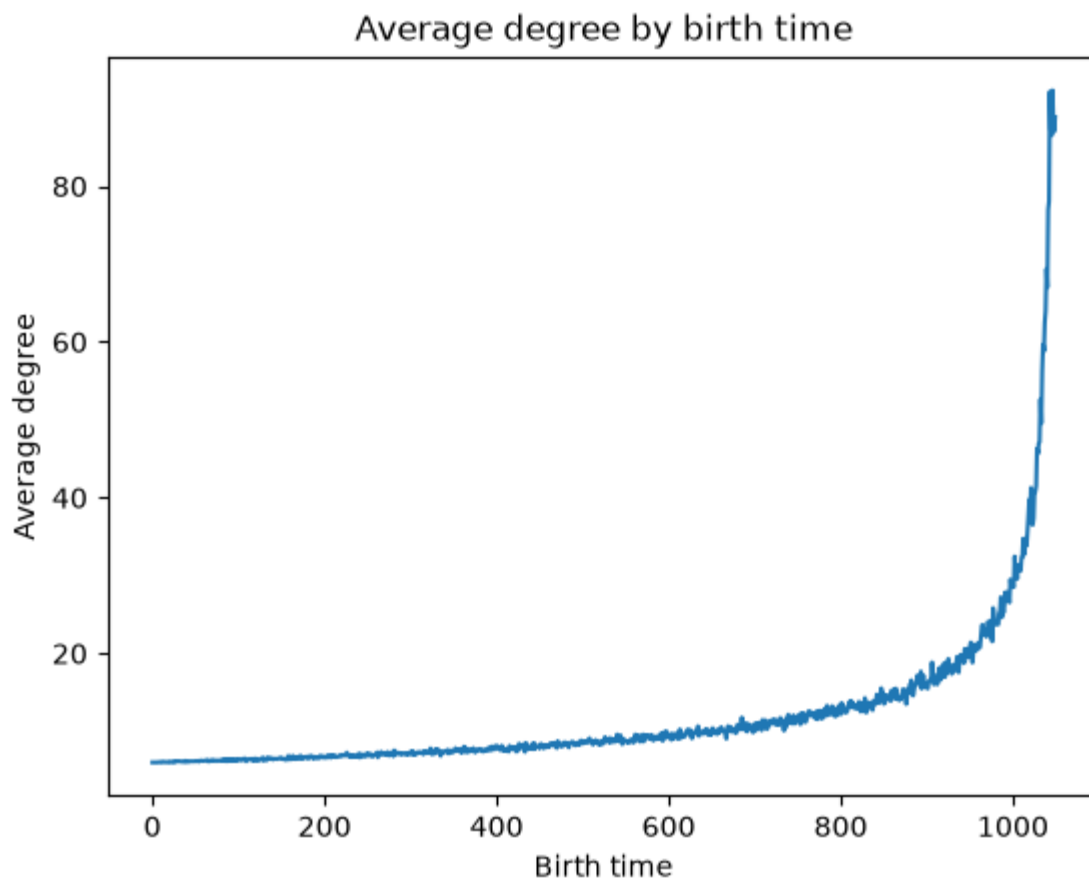
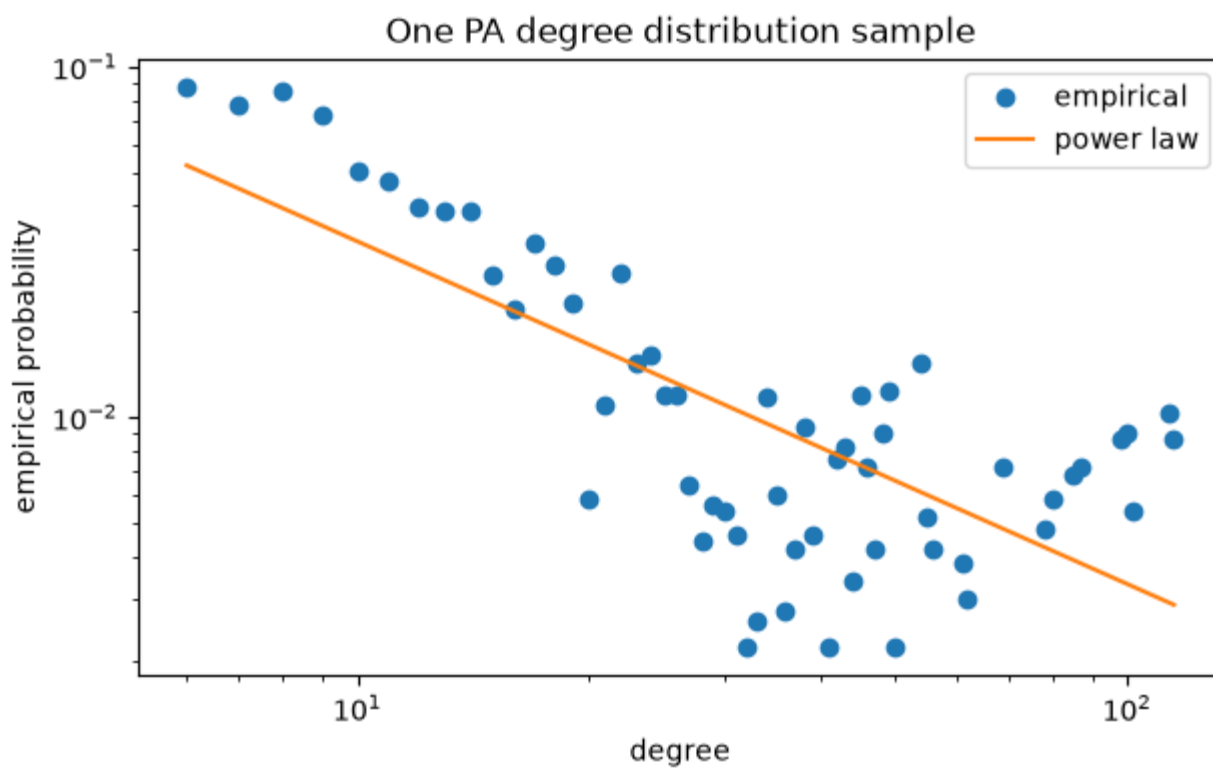


In []: question_2g(6)

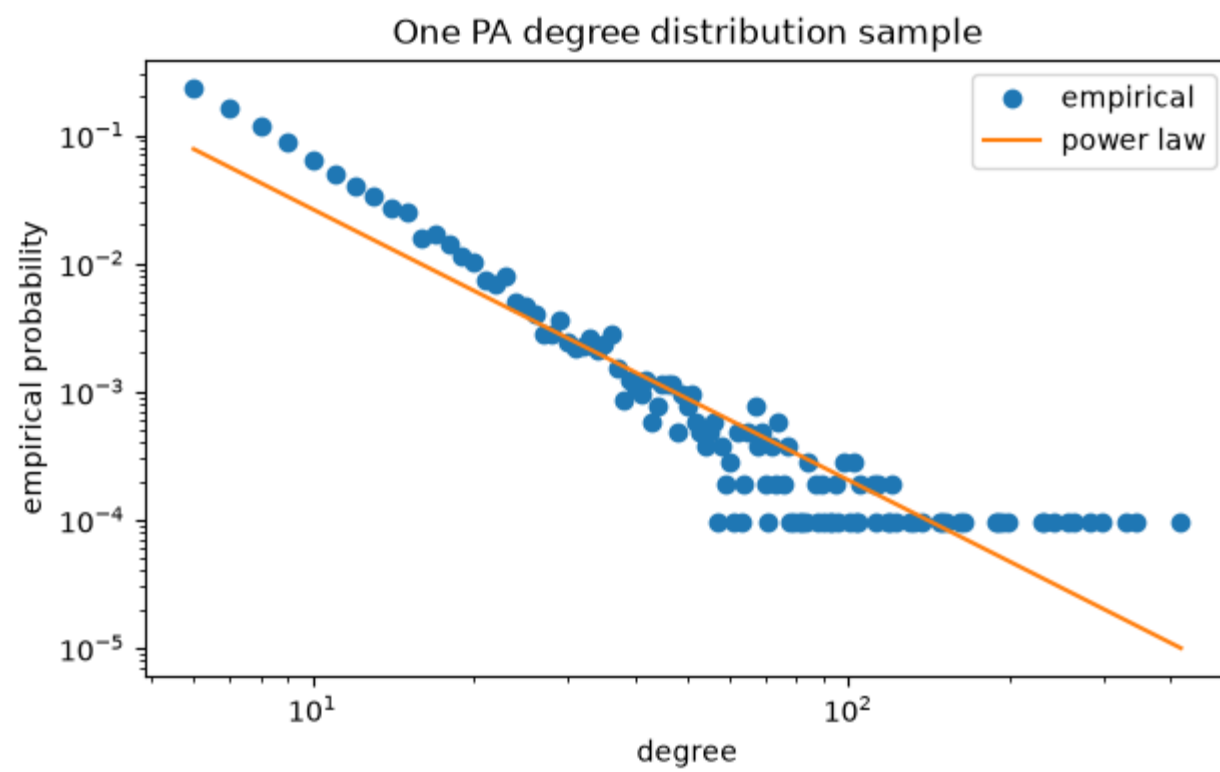
```
[True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True, True]
Yes, this network is always connected
IGRAPH U--- 1050 6279 --
connected: True
max degree: 119
mean degree: 11.96
degree variance: 126.4841142857143
edges: 6279 , assortativity: -0.012816169789175262 , modularity: 0.245
Nodes: 1050 , m = 6
rough log-log slope: -1.9576576502919065
```



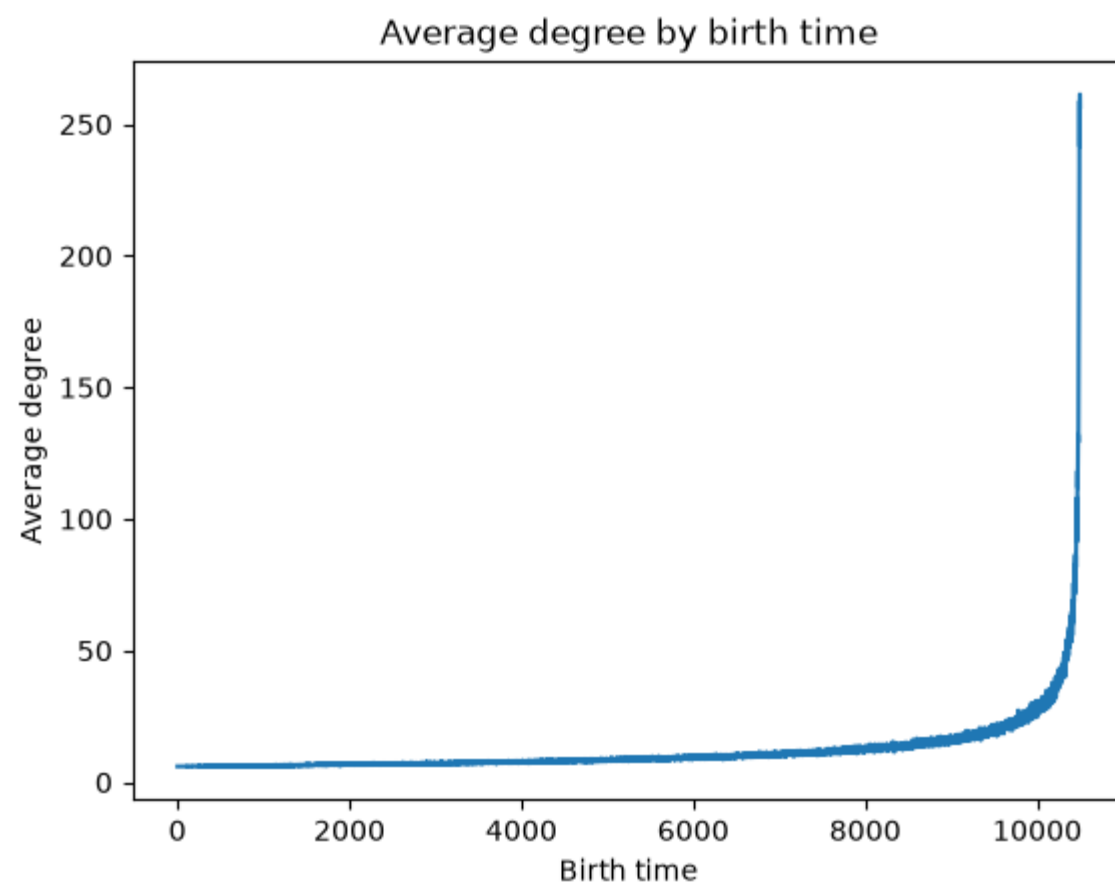
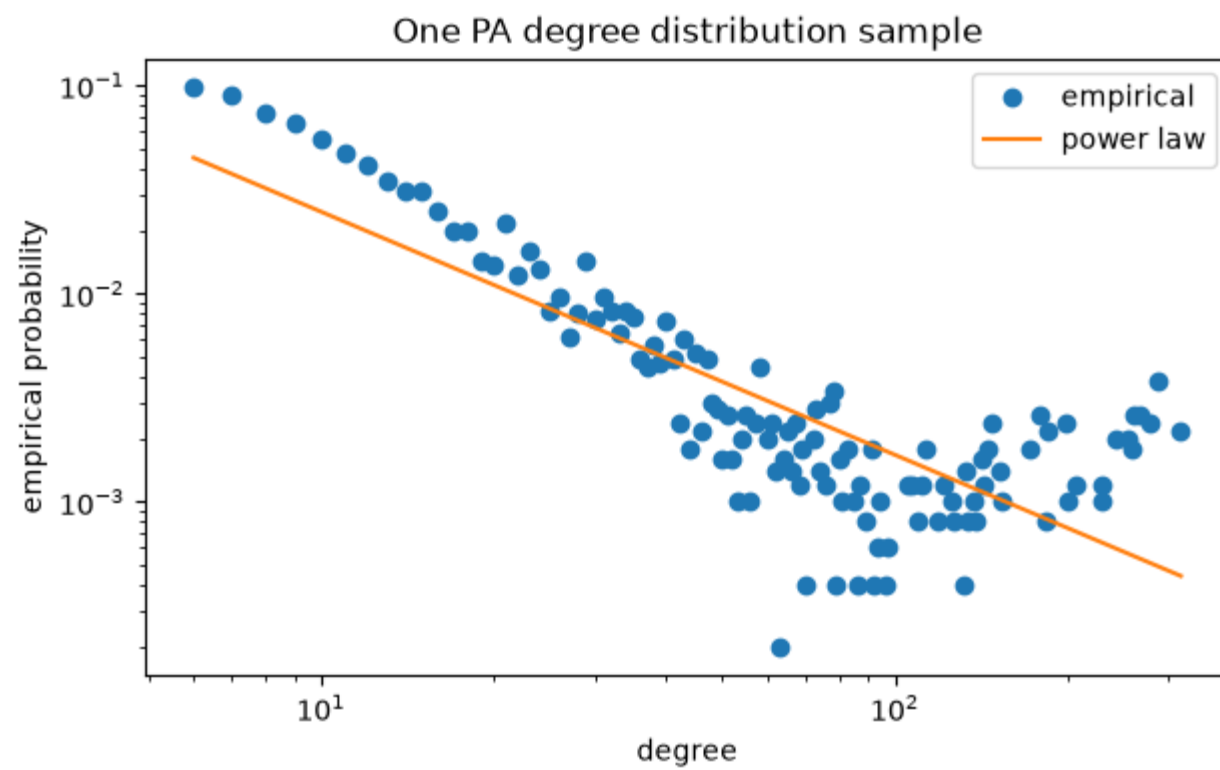
rough log-log slope: -0.9781989848926438



IGRAPH U--- 10500 62979 --
 connected: True
 max degree: 352
 mean degree: 11.996
 degree variance: 191.6180792380952
 edges: 62979 , assortativity: 0.0002519128383233273 , modularity: 0.25
 Nodes: 10500 , m = 6
 rough log-log slope: -2.1113083355986917



rough log-log slope: -1.1682994301888403



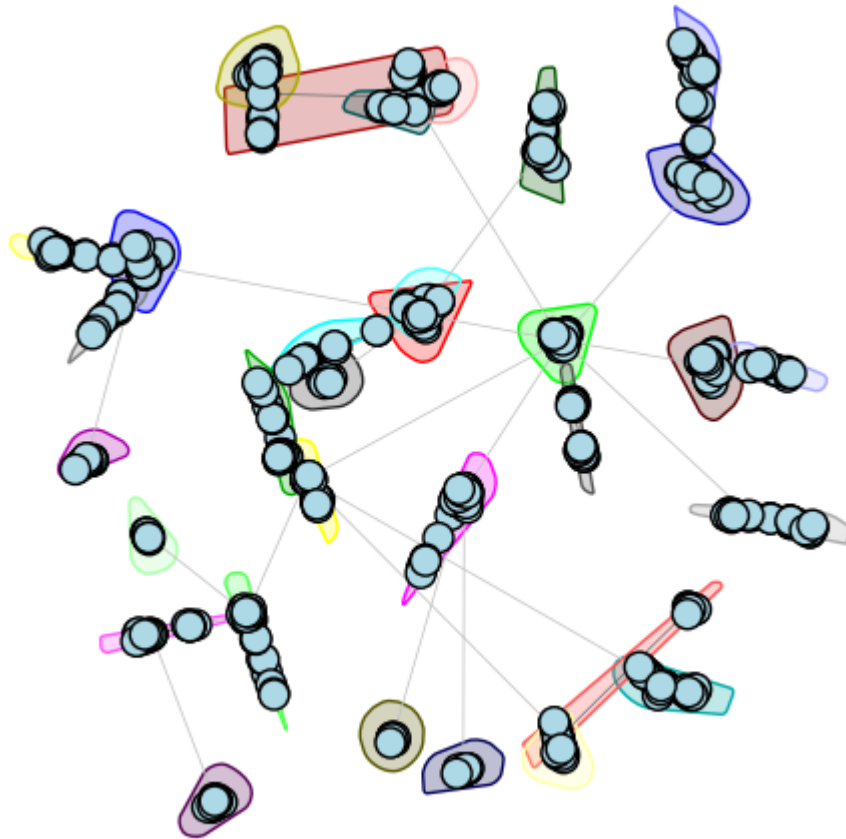
Main observations noted looking at PA networks with different values of m :

- Modularity decreases as m increases
- Assortativity value is close to 0 regardless of the value of m
- Mean degree increases as m increases
- Plotting the degree distribution of nodes in log-log scale still shows a linear distribution with a slope of around -2 regardless of the value of m
- Randomly picking a node i and getting the degree of nodes of one if its neighbor j , also shows a linear distribution with a less negative(or less steep) slope regardless of the value of m .

(h)

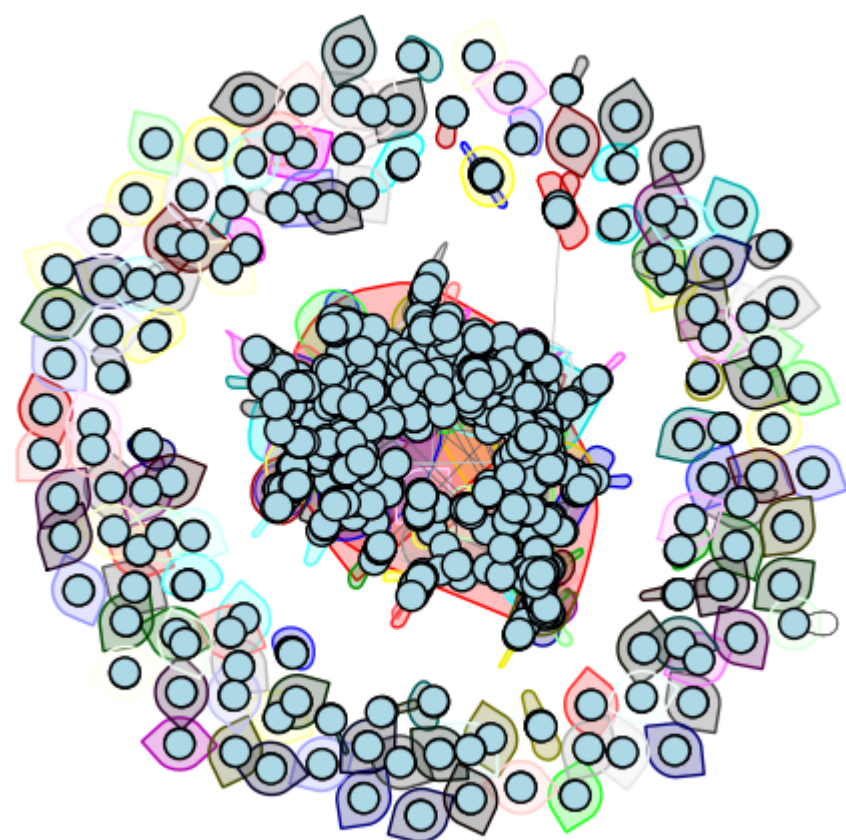
```
In [ ]: from project1_code import question_2h
question_2h()
```

Original Network
Used Fastgreedy
Modularity: 0.93



IGRAPH U--- 1050 1049 --
connected: True
max degree: 37
mean degree: 1.9980952380952381
degree variance: 6.457139229024944
edges: 1049 , assortativity: -0.09904526728278162

New Network
Fastgreedy failed (Error at src/community/fast_modularity.c:652: Fast greedy community detection works only on graphs without multi-edges. -- Invalid value), Using Walktrap
Modularity: 0.721



IGRAPH U--- 1050 1049 --
connected: False
max degree: 37
mean degree: 1.9980952380952381
degree variance: 6.457139229024944
edges: 1049 , assortativity: 0.03185862710335647

Comparison:

- Stub-matching network has a lower modularity than PA method

- Stub-matching network is also not connected whereas PA is
- PA network has a negative assortativity, while Stub-matching network is positive (and closer to 0)

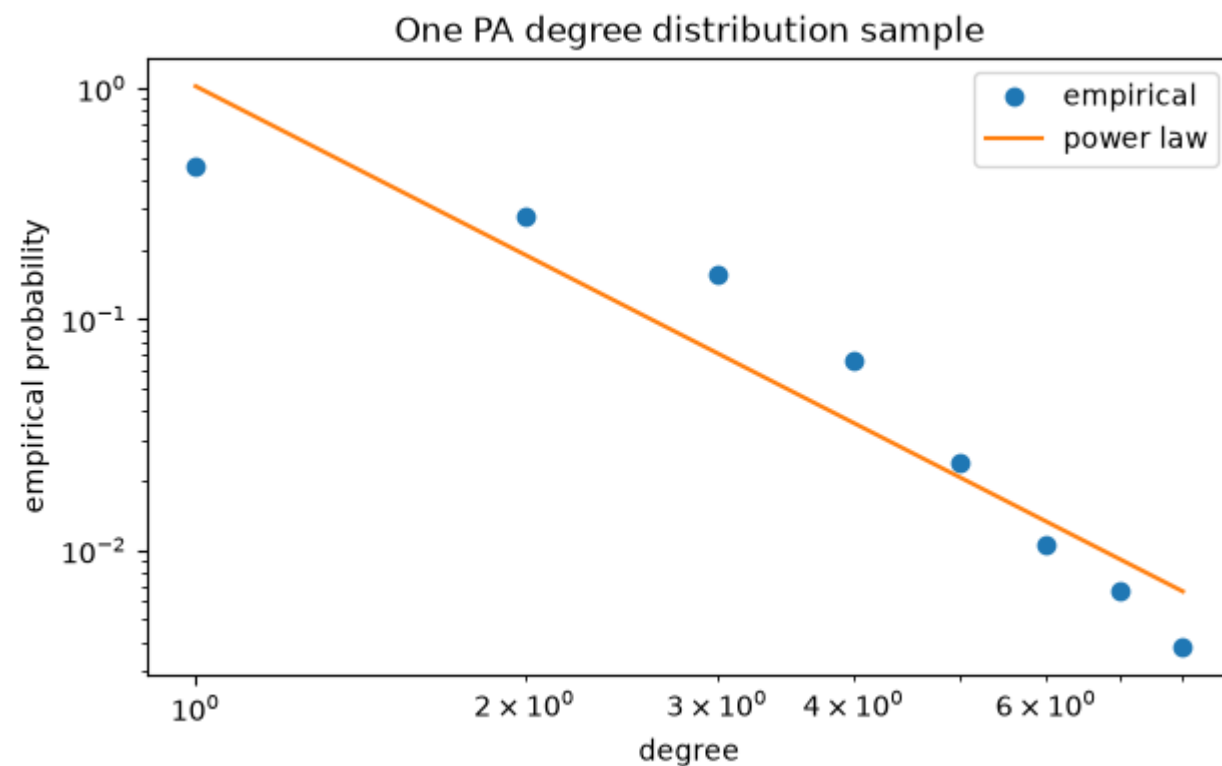
3. Create a modified preferential attachment model that penalizes the age of a node

```
In [ ]: from project1_code import question_3
question_3()
```

Part A

rough log-log slope: -2.421816596606091

Power Law Exponent is roughly: 2.421816596606091



Part B

Modularity: 0.936

2. Random Walk on Networks

```
In [ ]: from project1_code import question2_1
question2_1()
```

(a) Generate ER graph with $N = 900$

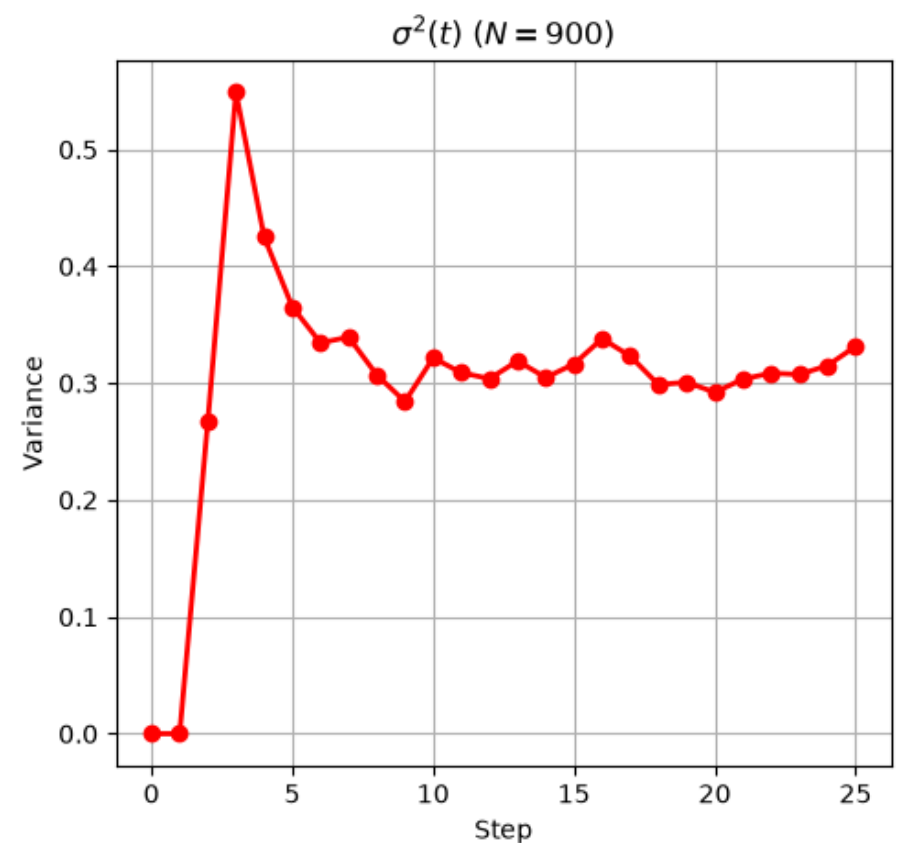
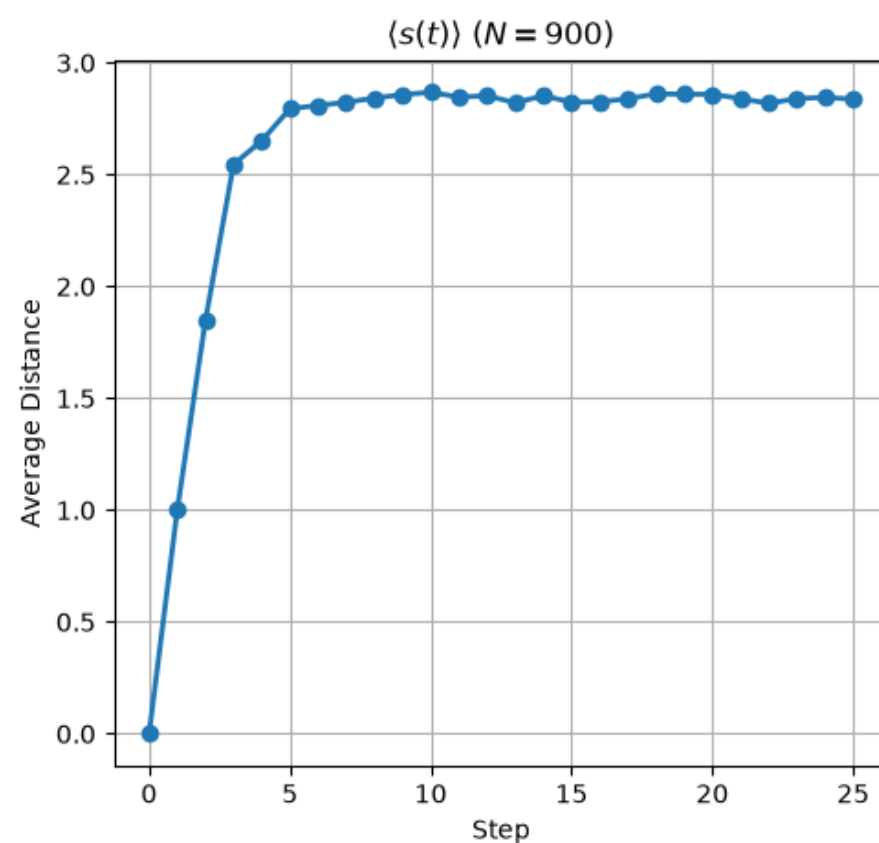
Nodes : 900

Edges : 6062

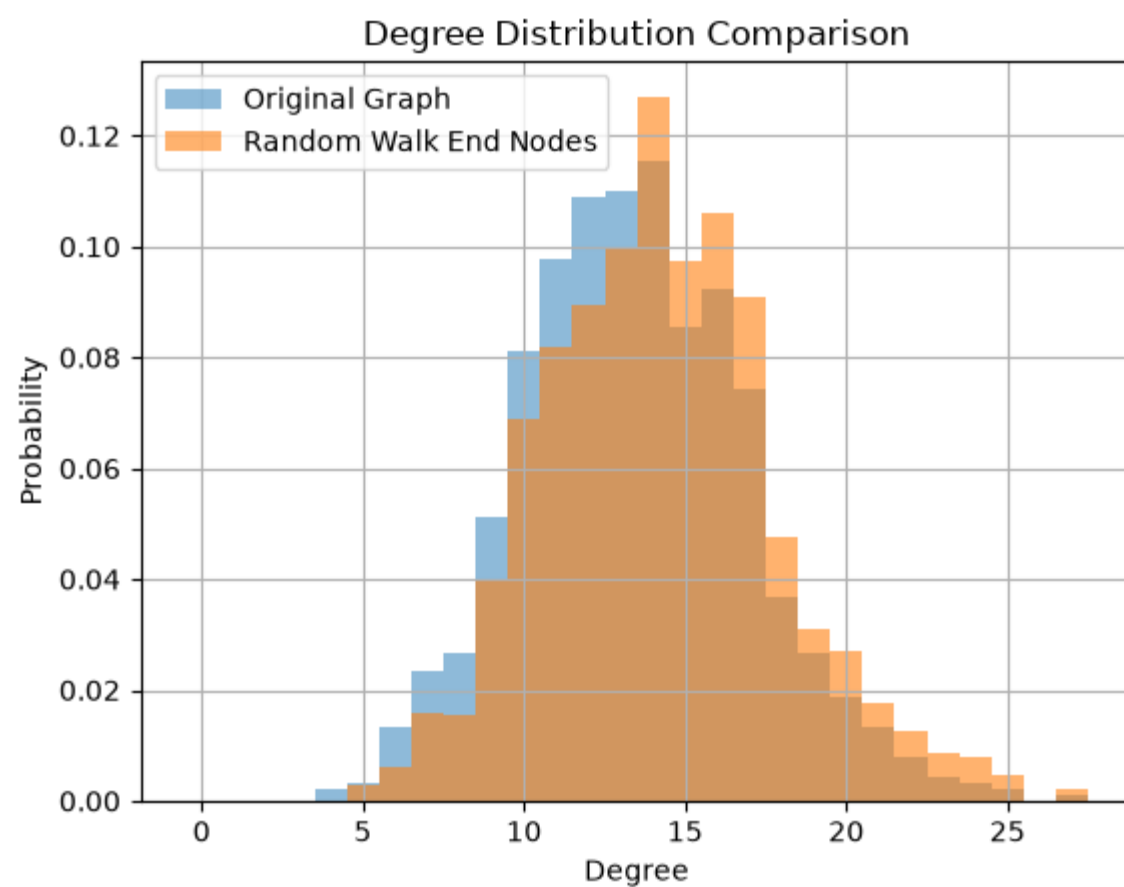
Diameter : 5

Simulating 2000 random walk trials

(b) Average distance and variance



(c) Degree distribution comparison

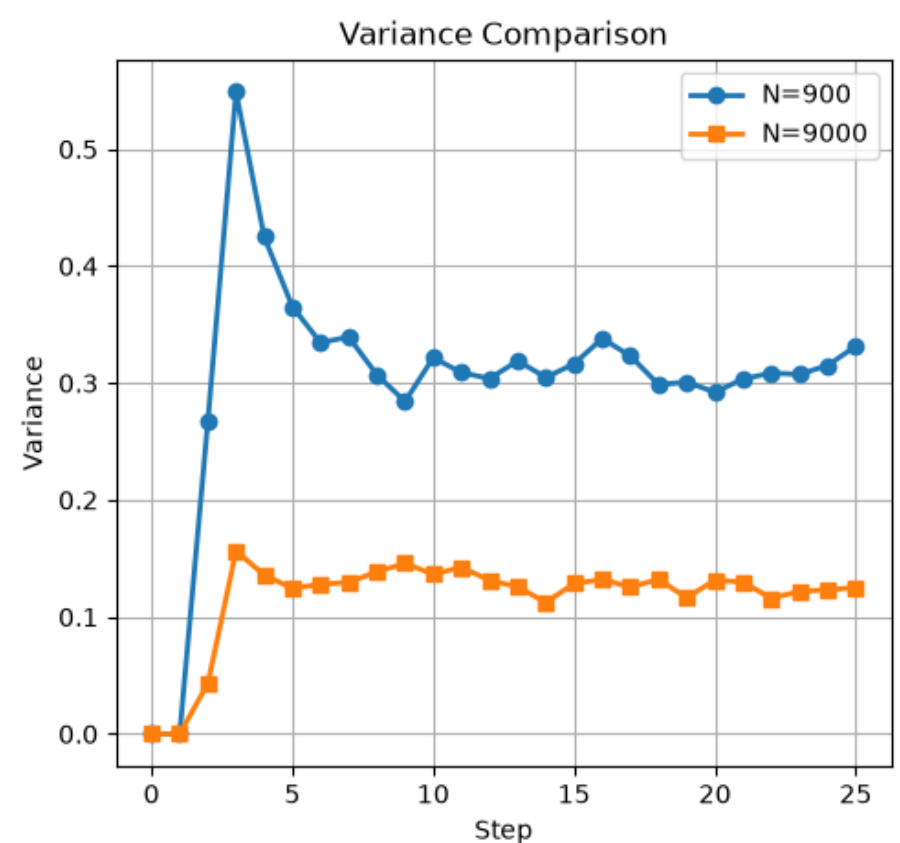
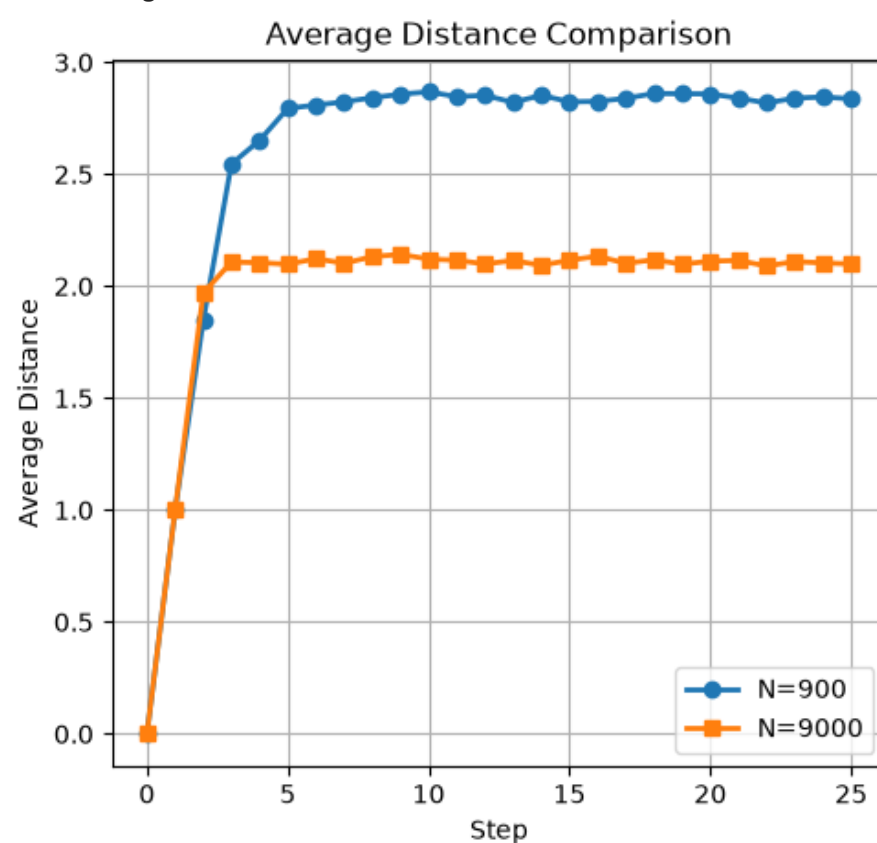


Average Graph Degree : 13.471
Average End-Node Degree : 14.266

The degree distribution of the nodes reached at the end of the random walk is biased toward higher-degree nodes compared to the degree distribution of the original graph. In an unbiased random walk on an undirected network, the probability of visiting a node in the long run is proportional to its degree, because nodes with more incident edges are more likely to be visited.

(d) Repeat for $N = 9000$

Nodes : 9000
Edges : 606089
Diameter : 3
Simulating 1000 random walk trials



Diameter Comparison

$N = 900$: 5

$N = 9000$: 3

When the network size was increased from 900 to 9000 nodes while keeping the edge probability fixed at $p=0.015$, the diameter decreased from 5 to 3. Although the larger graph contains ten times as many nodes, it is also much denser because the expected degree increases from approximately 13.5 to 135. The increased connectivity creates many additional shortcuts between nodes, reducing the maximum shortest-path distance in the network.

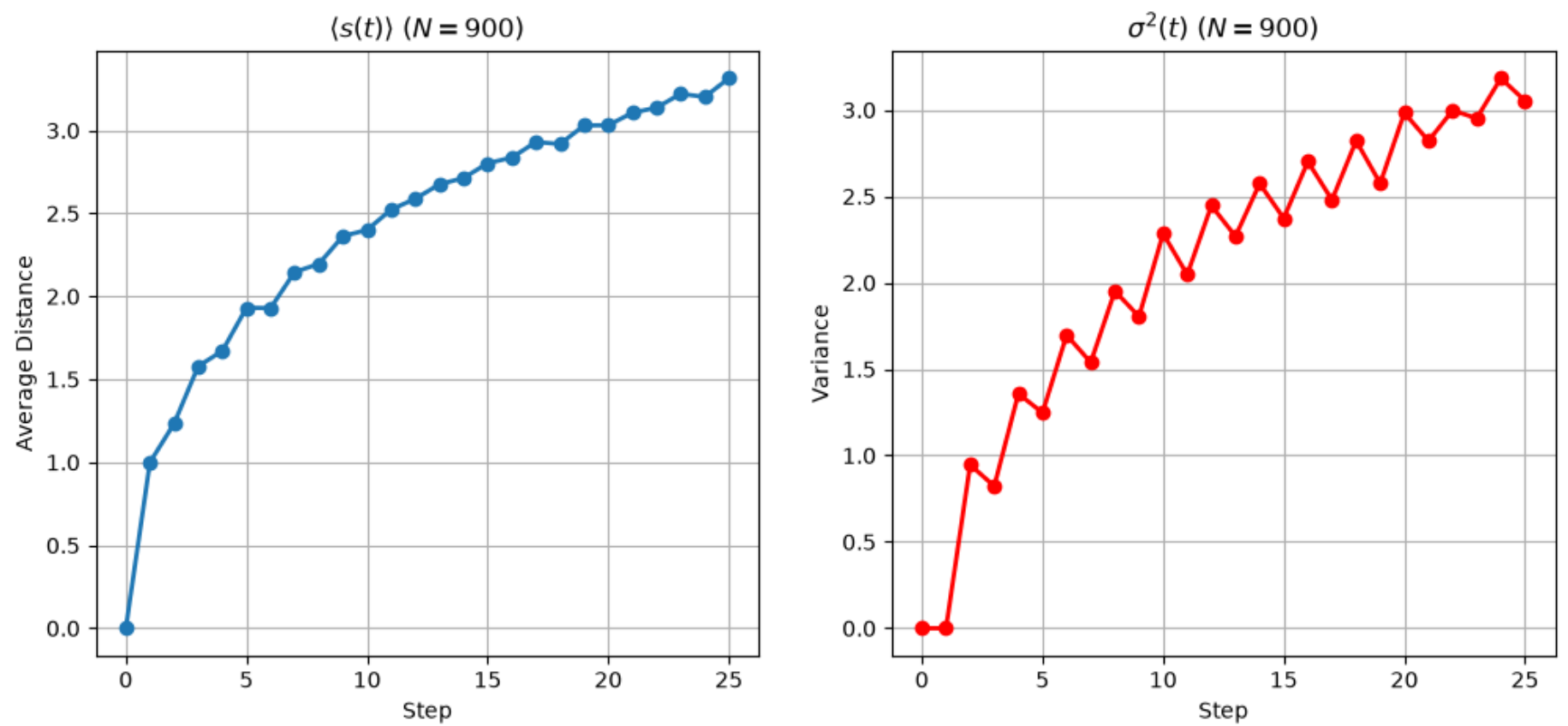
2. Random walk on networks with fat-tailed degree distribution

```
In [ ]: from project1_code import question2_2
question2_2()
```

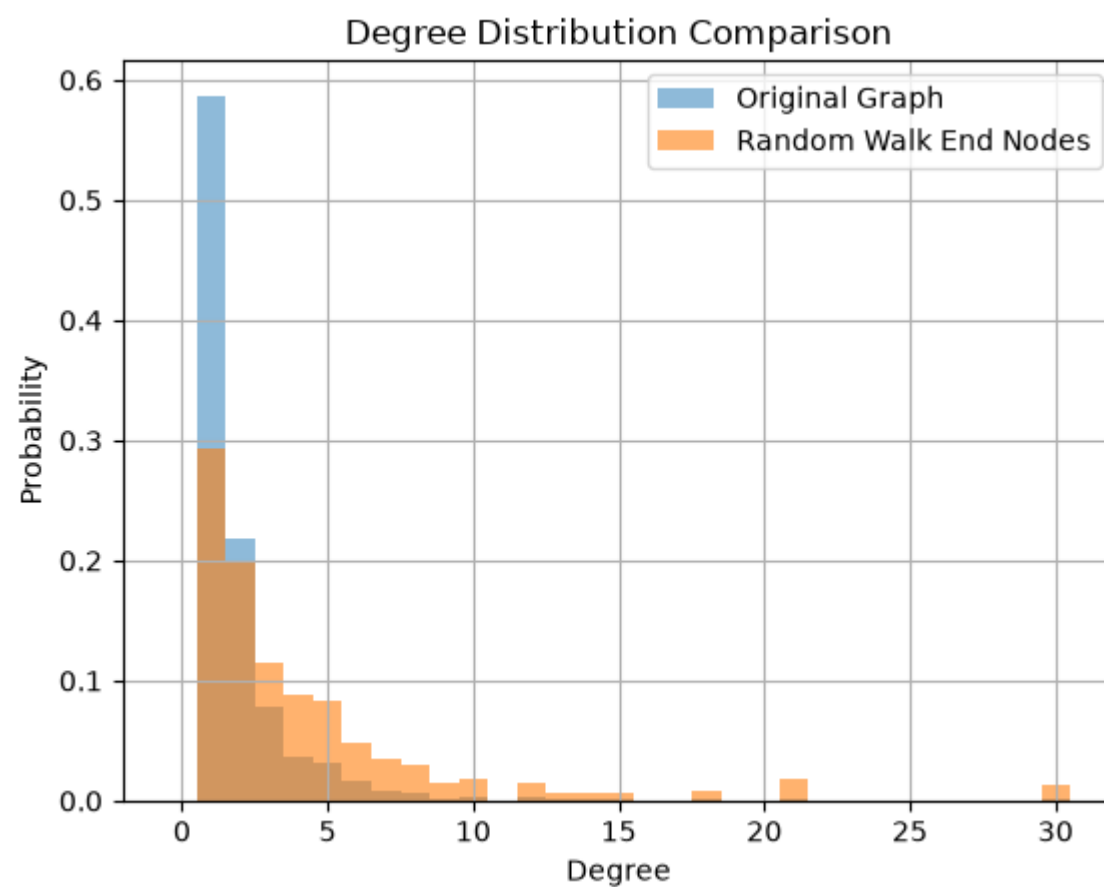
(a) Generate PA graph with $N = 900$, $M = 1$

Nodes : 900
Edges : 899
Diameter : 16
Simulating 2000 random walk trials

(b) Average distance and variance



(c) Degree distribution comparison



Average Graph Degree : 1.998

Average End-Node Degree : 4.300

With the original having an average degree of 1.998 and the random having 4.839, there is a large bias to higher degree nodes. This is an effect of having a stationary distribution on an undirected network where the probability of visit is directly proportional to its degree. In this graph the vast majority of nodes are leaves with a degree of 1 (over 60%), the random walker drops the probability of landing on degree-1 nodes to under 30%.

(d) Repeat for N = 9000

Nodes : 9000

Edges : 8999

Diameter : 28

Simulating 1000 random walk trials

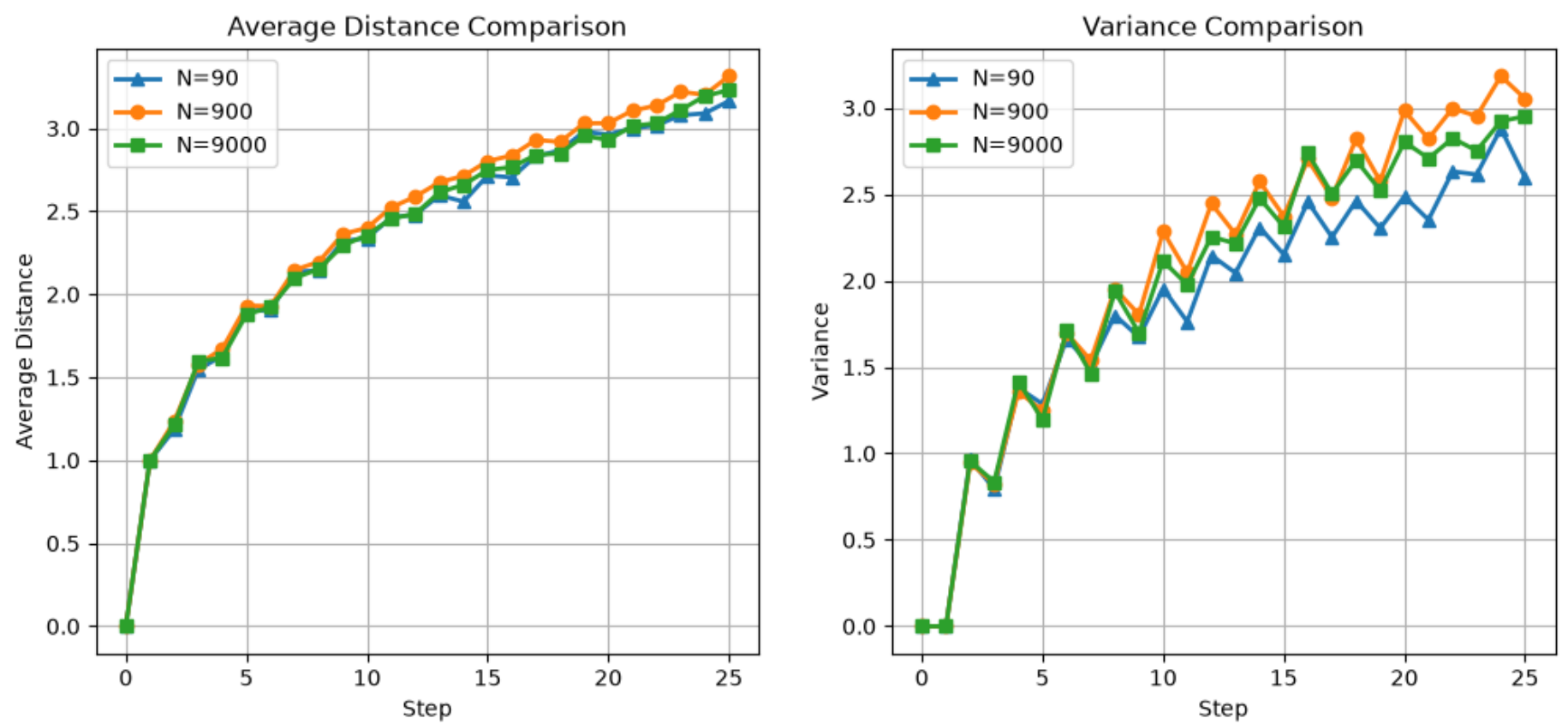
Repeat for N = 90

Nodes : 90

Edges : 89

Diameter : 9

Simulating 1000 random walk trials



Diameter Comparison

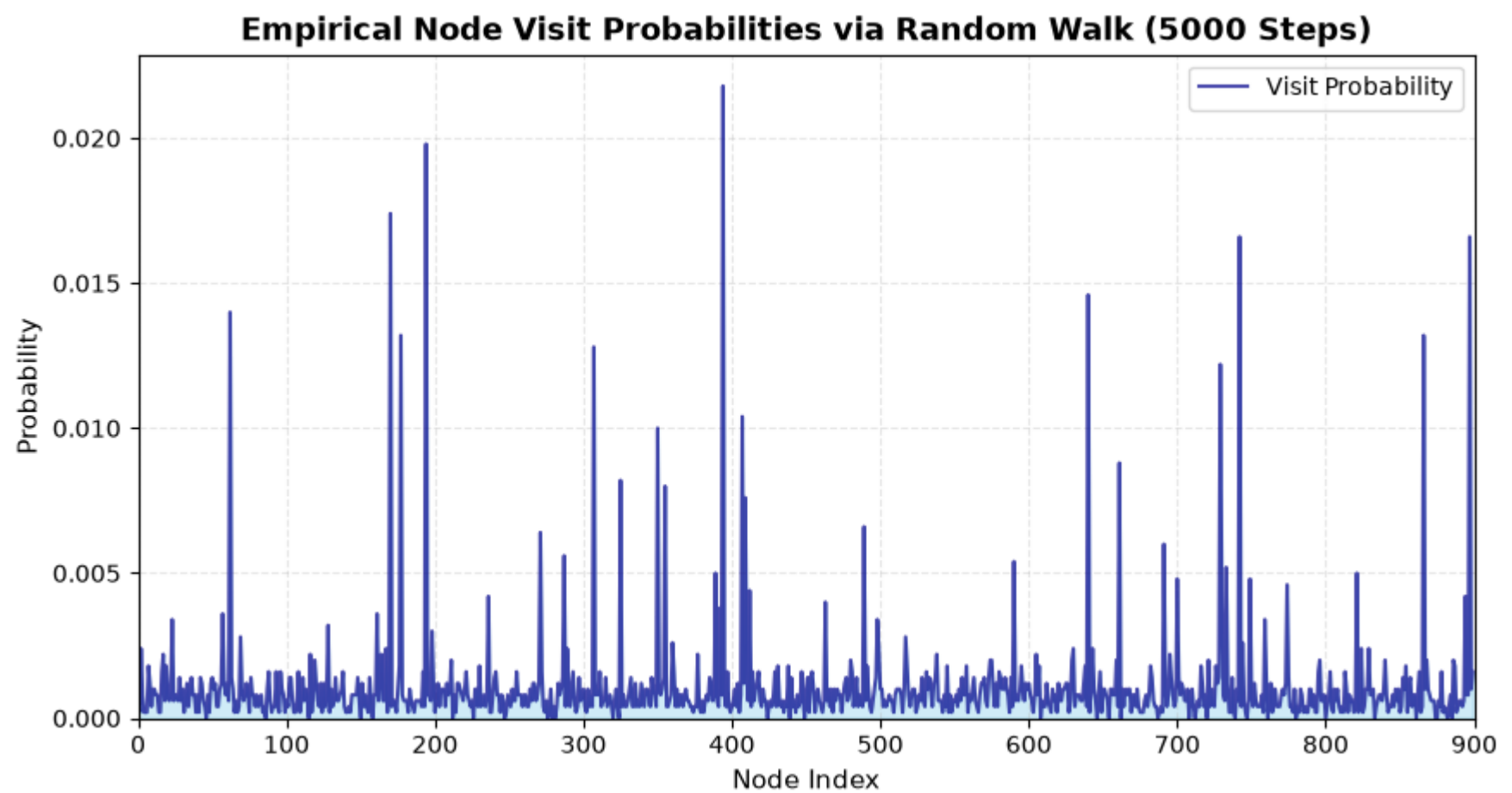
N = 90 : 9
N = 900 : 16
N = 9000 : 28

PA model fixes $m = 1$, locking the average degree at a sparse baseline of $k \approx 2$ for all network sizes. This network remains sparse as it grows, with the diameter increasing substantially with the number of nodes, expanding from 12 to 27 as grows 90 to 9000, shifting also the avg distance and variance. The expanding diameter also causes the random walker significantly more steps to explore the larger periphery and the distance keeps increasing showing that the system has not yet reached global equilibrium.

3. PageRank

(a)

```
In [28]: from project1_code import question2_3a
question2_3a()
```



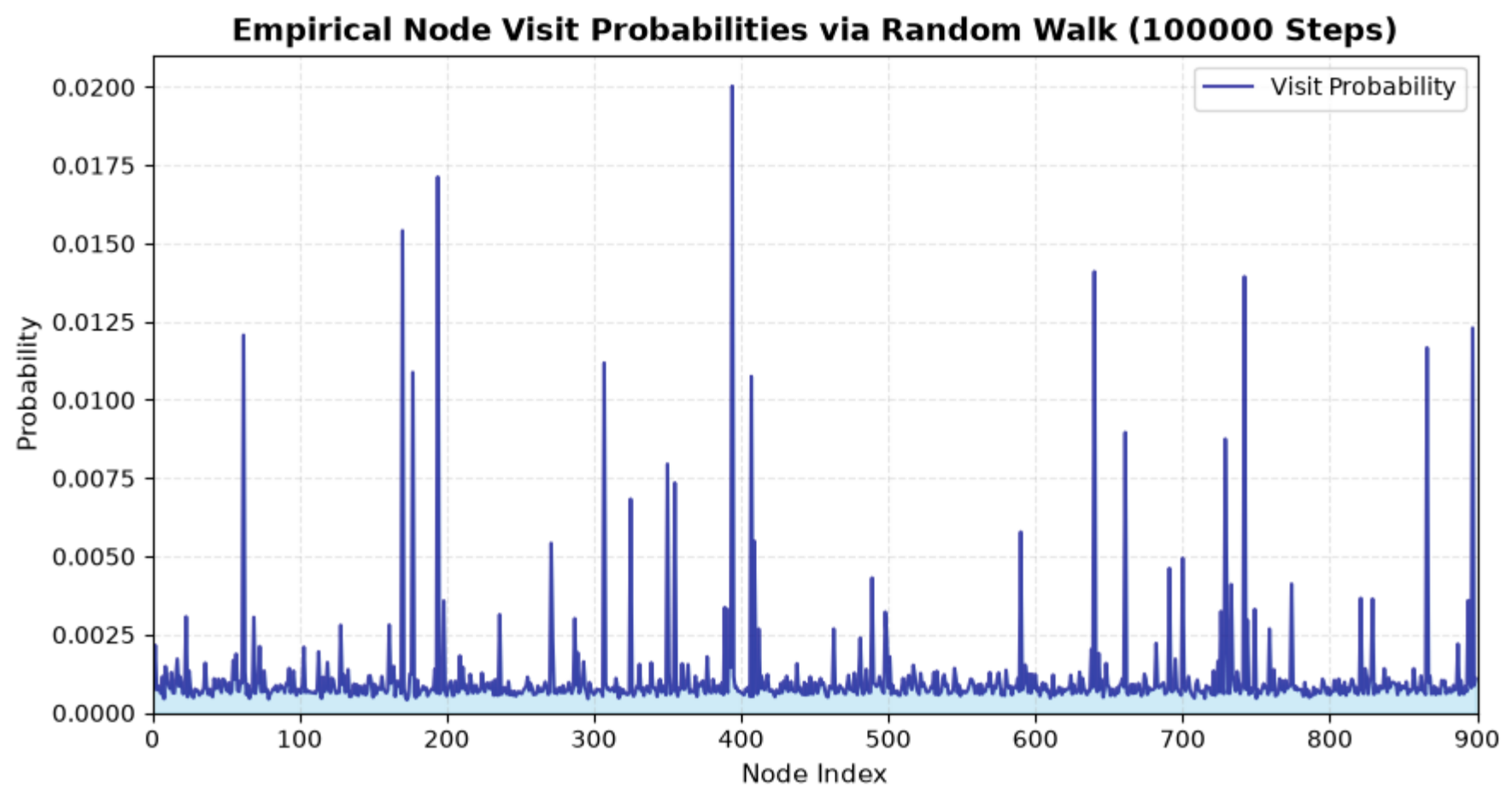
Node with highest in-degree: 394

Node with highest visit probability: 394

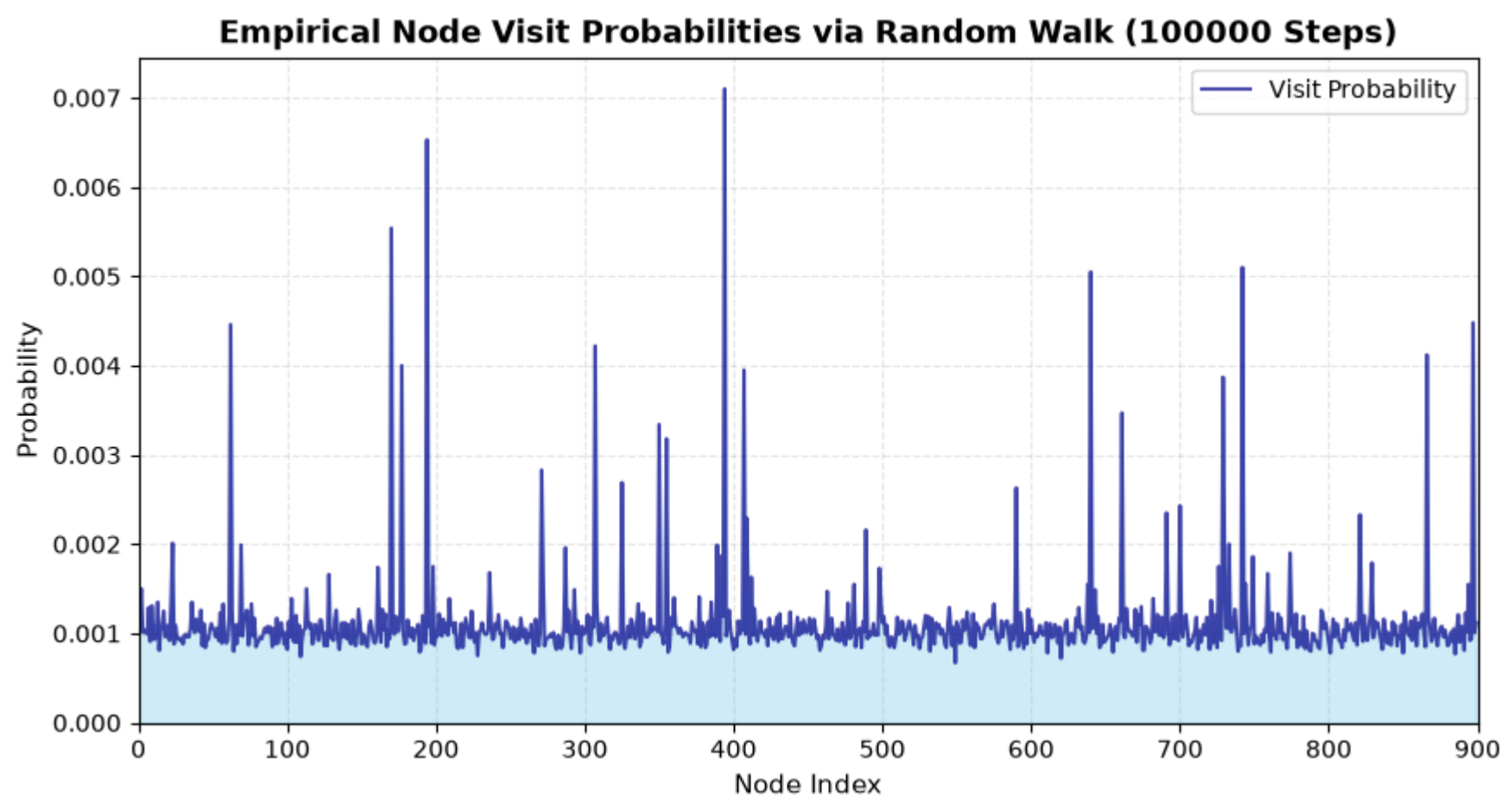
The plot above shows that the visit probability is proportional to the in-degree of a node (i.e., number of edges directed into a node). Over a long random walk of 5,000 steps, we observed that the node with the highest in-degree had the highest visit probability.

(b)

```
In [29]: from project1_code import question2_3b
question2_3b(alpha=0.2, num_steps=int(1e5)) # Lower alpha value
```

```
In [30]: # Higher alpha value
question2_3b(alpha=0.8, num_steps=int(1e5))
```



For PageRank with teleportation, when α (i.e., the probability of teleportation) is low, the network's node degree dictates a node's visit probability, meaning the preferential hubs will have the highest visit probabilities. As shown in the first plot above, there are spikes in the probabilities around the preferential hubs.

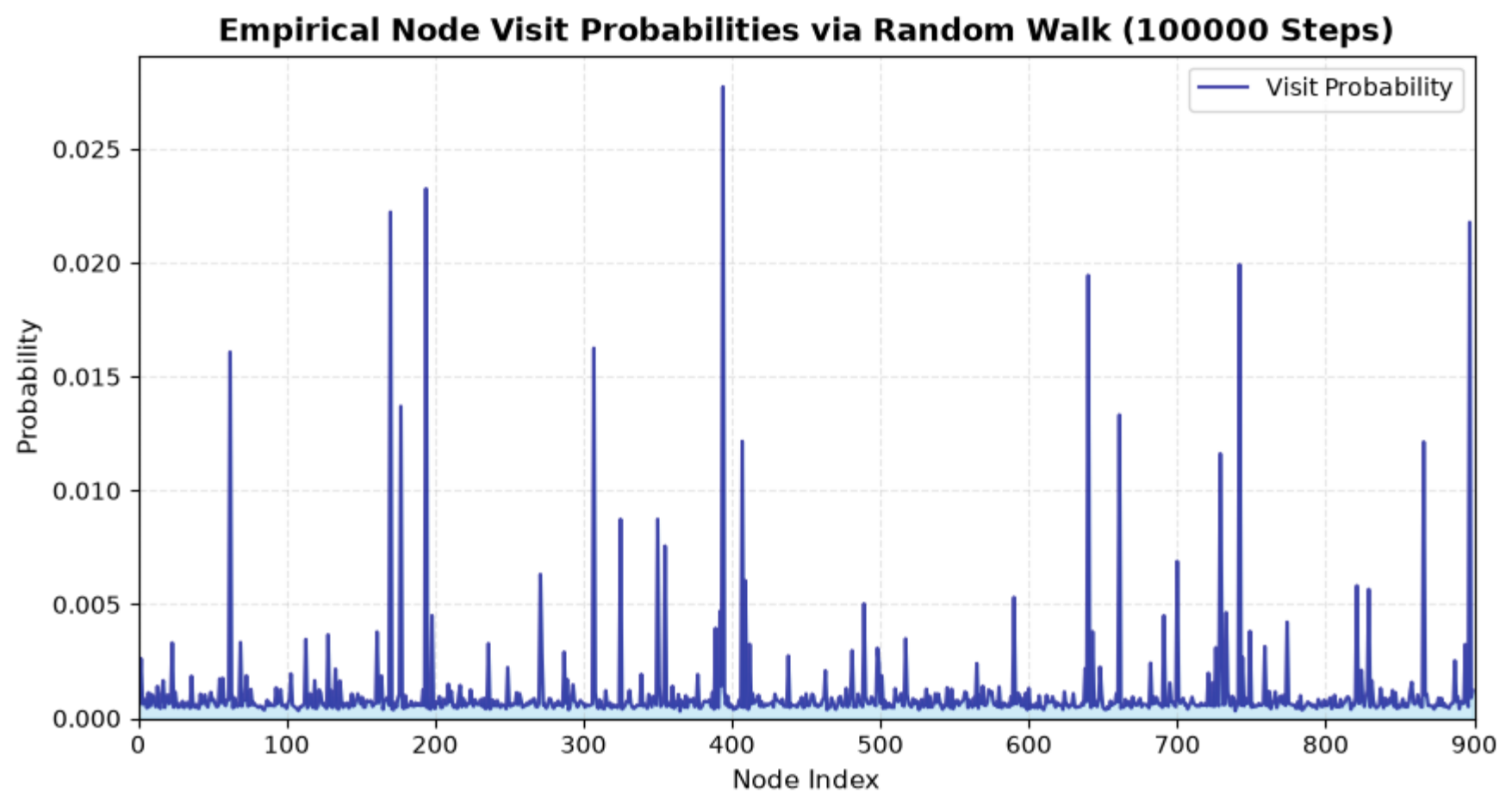
When α is high, however, the walker teleports constantly and the structure of the network is essentially ignored as the walker is teleported rather than following directed edges. The probability essentially flattens out to a uniform distribution. With 900 nodes, the uniform probability is $1/900 = 0.001$, which is reflected in the higher floor centered around this value in the second graph above.

4. Personalized PageRank

(a)

```
In [31]: from project1_code import question2_4a
question2_4a(alpha=0.2, num_steps=int(1e5)) # Lower alpha value
```

Teleport probabilities sum: 1.0000



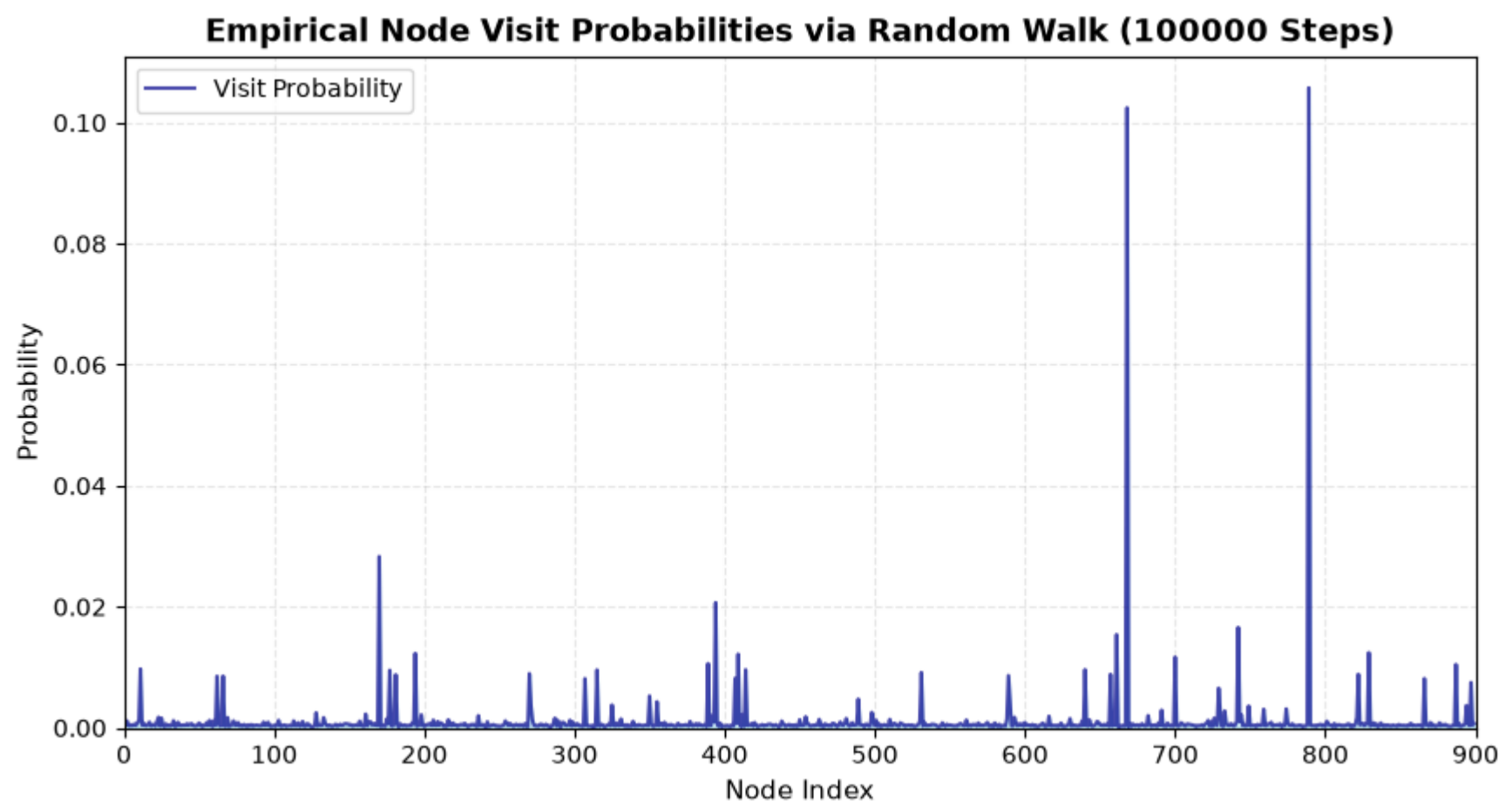
Node with highest in-degree: 394
Node with highest visit probability: 394

Because the teleportation probability $\alpha=0.2$ is low, the Personalized PageRank results here (with custom teleport probabilities) are similar to our 3(a) results. Since the walker doesn't teleport often, the empirical node probabilities are similar in both problems, since the walker generally follows the directed edges in the network.

(b)

```
In [32]: from project1_code import question2_4b
question2_4b(alpha=0.2, num_steps=int(1e5)) # Lower alpha value
```

Node 789 has a middle PageRank score of: 0.000272
Node 668 has a middle PageRank score of: 0.000273
The median score is: 0.000272
Median nodes: [np.int64(789), np.int64(668)]



Node with highest in-degree: 394
Node with highest visit probability: 789

Because the two median nodes are the only two teleportation hubs, everytime the walker teleports, it starts at those two nodes, and the PageRank values are highest there. The empirical probabilities elsewhere are as a result of the walker moving to connected edges from those hubs.

(c)

The general PageRank equation

$$\pi(i) = (1 - \alpha) \sum_{j=1}^n \left(\frac{1}{k_{\text{out}}(j)} A_{ji} \pi(j) \right) + \frac{\alpha}{n}$$

transforms to

$$\pi(i) = (1 - \alpha) \sum_{j=1}^n \left(\frac{1}{k_{\text{out}}(j)} A_{ji} \pi(j) \right) + \frac{\alpha}{|T|}$$

where T is a set of trusted nodes. If the node i is not a trusted node, the personalized PageRank equation reduces to

$$\pi(i) = (1 - \alpha) \sum_{j=1}^n \left(\frac{1}{k_{\text{out}}(j)} A_{ji} \pi(j) \right)$$